



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ

Ηλίας Ευαγγέλου, Αικατερίνη Γερογιαννοπούλου, Κάργας Στέφανος
(el23073), (el23218), (el23059)

Βάσεις Δεδομένων
6ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

Εξαμηνιαία Εργασία

Αθήνα 2026

Περιεχόμενα:

1. Εισαγωγή
2. Μετάφραση του προβλήματος στη Βάση δεδομένων, ER Model
3. Παραδοχές Σχεδιασμού
4. Relational Schema
5. Indexes και Βελτιστοποίηση
6. Triggers
7. Stored Procedures
8. Views
9. Data Generation And Loading
10. Queries
11. Χρήση AI
12. Εγκατάσταση Βάσης
13. Repository Structure

[illegible]

2.1. Ανθρώπινο Δυναμικό (HR) και Ιεραρχία

Το σύστημα υιοθετεί τη φιλοσοφία της κληρονομικότητας. Η κεντρική οντότητα Personnel (Προσωπικό) περιλαμβάνει τα κοινά στοιχεία όλων των εργαζομένων (ΑΜΚΑ, στοιχεία επικοινωνίας). Οι εξειδικευμένες οντότητες Doctor (Γιατρός), Nurse (Νοσηλεύτης) και Administrative_staff (Διοικητικό Προσωπικό) συνδέονται με το προσωπικό, επιτρέποντας την αποθήκευση ειδικών χαρακτηριστικών (π.χ. άδεια ασκήσεως επαγγέλματος για γιατρούς, βαθμίδα για νοσηλεύτες) χωρίς πλεονασμό δεδομένων. Τέλος υπάρχει η οντότητα Department (Τμήμα), όπου κάθε τμήμα διοικείται από έναν διευθυντή-γιατρό.

2.2. Υποδομές και Διαχείριση Χώρων

Η οργάνωση του φυσικού χώρου βασίζεται στις οντότητες Department, Bed (Κλίνη) και Operating_place (Χώρος Επεμβάσεων). Η φιλοσοφία εδώ είναι ο έλεγχος της διαθεσιμότητας: κάθε κλίνη και χειρουργείο παρακολουθείται μέσω των status (Available, Occupied, Maintenance), διασφαλίζοντας ότι δεν θα γίνει διπλοκράτηση ή χρήση μη λειτουργικών υποδομών.

2.3. Ο Ασθενής

Ο ασθενής αποτελεί το κέντρο του συστήματος. Η οντότητα Patient (Ασθενής) αποθηκεύει το ιστορικό και τα δημογραφικά στοιχεία, καθώς και στοιχεία ανάγκης με το entity Emergency Contact. Η είσοδος στο νοσοκομείο μοντελοποιείται μέσω της οντότητας Emergency_visit, η οποία χρησιμοποιεί το σύστημα διαλογής (triage), ενώ η Hospitalization (Νοσηλεία) αποτελεί την κεντρική οντότητα-γεγονός που συνδέει τον ασθενή με το τμήμα, το κρεβάτι, τους θεράποντες ιατρούς και το τελικό κόστος όπως φαίνεται από τις σχέσεις στο ER. Για την παροχή φαρμάκων υπάρχουν οι οντότητες Prescription και Medication αντίστοιχα για συνταγογράφηση και αγωγή.

2.4. Ιατρικά Πρωτόκολλα και Ασφάλεια

Χρησιμοποιούνται διεθνή και Ελληνικά πρότυπα:

1. ICD10_diagnosis: Τυποποιημένη κωδικοποίηση ασθενειών για ακριβή διάγνωση.
2. KEN (Κλειστά Ενοποιημένα Νοσήλια): Μοντέλο κοστολόγησης που συνδέει την ιατρική πράξη με την οικονομική διαχείριση.

Drug & Active_substance: Η φιλοσοφία διαχωρισμού του εμπορικού φαρμάκου από τη δραστική ουσία είναι κρίσιμη για την ασφάλεια, καθώς οι αλλεργίες των ασθενών (Patient_allergy) καταγράφονται σε επίπεδο ουσίας αφού κάθε φάρμακο μπορεί να περιέχει πολλές ουσίες.

2.5. Κατάλογοι Εξετάσεων και Επεμβάσεων

Οι οντότητες Lab_test_catalog και Procedure_catalog λειτουργούν ως λεξικά υπηρεσιών. Κάθε εξέταση (Lab_test) ή επέμβαση (Procedure_event) που πραγματοποιείται πρέπει να αντιστοιχεί

σε μια προκαθορισμένη εγγραφή του καταλόγου, επιβάλλοντας ομοιομορφία στις ιατρικές αναφορές και τη χρήση των χώρων.

2.6. Ποιότητα Υπηρεσιών και Πολυμέσα

Η οντότητα `Hospitalization_evaluation` επιτρέπει στους ασθενείς να αξιολογούν τις παροχές (φαγητό, καθαριότητα, ιατρική φροντίδα). Τέλος, το σχήμα περιλαμβάνει μια ευέλικτη δομή διαχείρισης εικόνων (`Image_asset` και `Entity_image`), η οποία επιτρέπει τη σύνδεση οπτικού υλικού με οποιαδήποτε οντότητα της βάσης.

3. Παραδοχές Σχεδιασμού

3.1. Το νοσοκομείο έχει σταθερή δομή τμημάτων. Πιο συγκεκριμένα, δεν παράγουμε δυναμικά νέα τμήματα. Θεωρούμε ότι τα βασικά 15 τμήματα του νοσοκομείου είναι σταθερά για την περίοδο μελέτης.

3.2. Κάθε προσωπικό έχει έναν βασικό τύπο. Δηλαδή κάθε άτομο ανήκει σε μία από τις βασικές κατηγορίες: ιατρός, νοσηλεύτης ή διοικητικός. Αυτό αποφεύγει διπλούς ρόλους στο ίδιο άτομο.

3.3. Οι ιατροί μπορούν να συνδέονται με περισσότερα από ένα τμήματα. Στην ουσία υπάρχει πίνακας `doctor_department`, άρα ένας ιατρός μπορεί να συνεργάζεται με δεύτερο τμήμα. Παρ' όλα αυτά έχει μία βασική ειδικότητα και μία ιεραρχική θέση.

3.4. Η ιεραρχία ιατρών είναι δενδρική. Αυτό σημαίνει ότι κάθε ιατρός, εκτός από διευθυντή, έχει έναν επόπτη. Θεωρούμε ότι δεν επιτρέπονται κύκλοι στην εποπτεία και ότι ο διευθυντής βρίσκεται στην κορυφή.

3.5. Η νοσηλεία ανήκει σε ένα τμήμα και ένα κρεβάτι και άρα μια νοσηλεία καταγράφεται σε ένα κύριο τμήμα και ένα κρεβάτι. Δεν μοντελοποιούμε μεταφορές ασθενή μεταξύ τμημάτων μέσα στην ίδια νοσηλεία.

3.6. Το συνολικό κόστος νοσηλείας βασίζεται στο KEN. Δηλαδή το κόστος υπολογίζεται ως βασικό κόστος KEN συν πρόσθετη χρέωση όταν η διάρκεια υπερβαίνει τη μέση διάρκεια νοσηλείας. Αυτή είναι απλοποίηση του πραγματικού billing.

3.7. Οι ICD-10 και KEN κωδικοί χρησιμοποιούνται ως επίσημα reference data. Δεν εφευρίσκουμε κωδικούς διάγνωσης ή KEN. Τα synthetic δεδομένα συνδέονται πάνω σε καθαρισμένα official/reference CSV.

3.8. Τα φάρμακα και οι δραστικές ουσίες βασίζονται σε official EMA data. Πιο συγκεκριμένα τα φάρμακα, οι δραστικές ουσίες και οι σχέσεις τους θεωρούνται reference data. Οι συνταγές και αλλεργίες είναι synthetic αλλά παράγονται μόνο με βάση αυτές τις επίσημες ουσίες.

3.9. Οι βάρδιες είναι αντιπροσωπευτικό ιστορικό δείγμα. Για πρακτικούς λόγους, δεν κρατάμε κάθε ημέρα τριετίας με όλες τις βάρδιες. Κρατάμε πλήρη εβδομάδα ανά μήνα σε τριετές εύρος, ώστε να υπάρχει ιστορικό βάθος χωρίς υπερβολικό load time.

3.10. Οι βάρδιες γίνονται πρώτα PROCESSING και μετά μπορούν να γίνουν VALID. Η κατάσταση VALID σημαίνει ότι πέρασαν οι κανόνες κάλυψης προσωπικού και επίβλεψης ειδικευόμενων.

3.11. Τα triggers υλοποιούν business rules που δεν καλύπτονται καλά με απλά CHECK. Π.χ. αποφυγή επικάλυψης επεμβάσεων, αλλεργίες σε συνταγογραφήσεις, όρια βαρδιών ανά μήνα, ανάπαυση μεταξύ βαρδιών, επίβλεψη ειδικευόμενων.

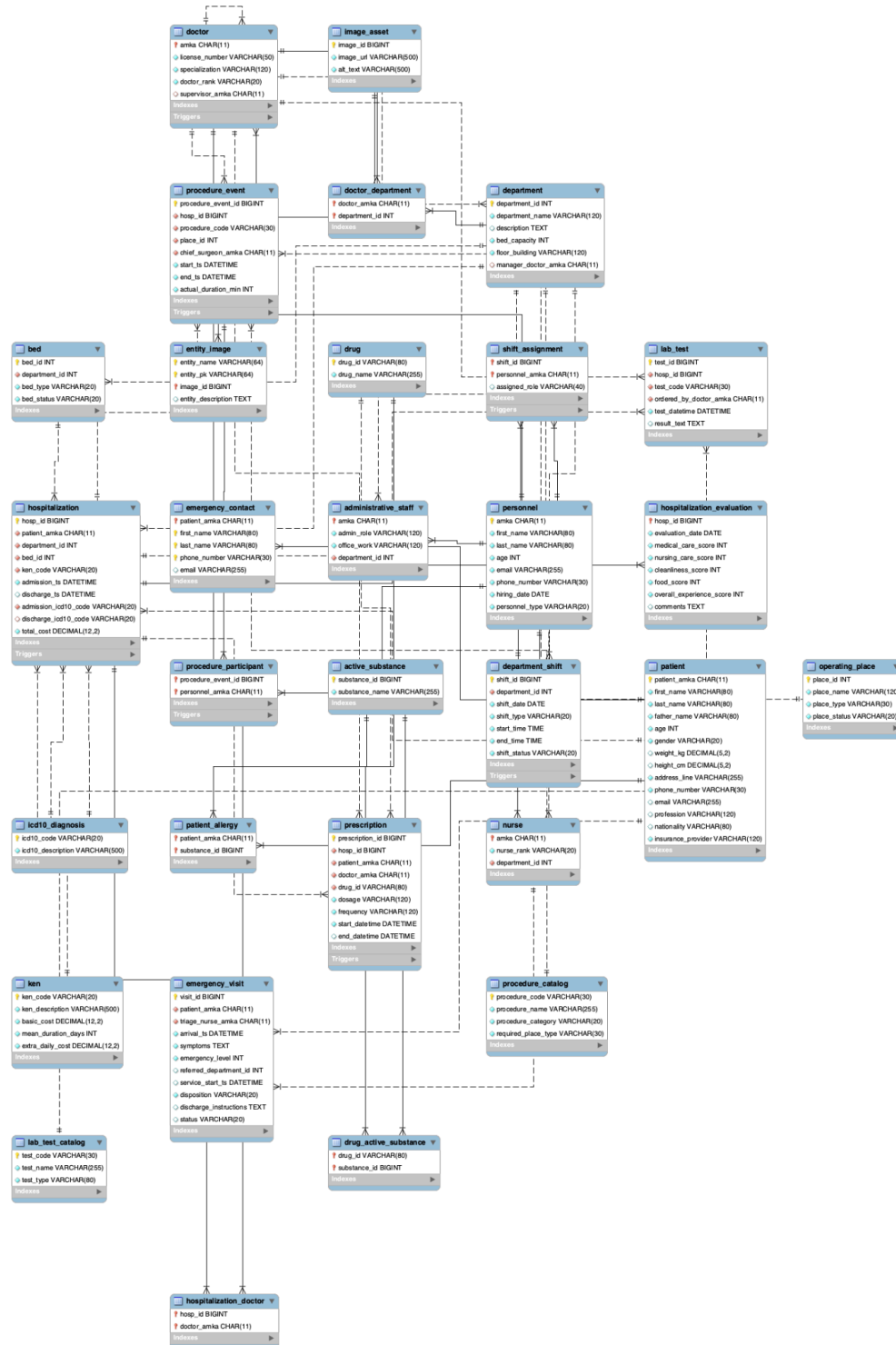
3.12. Οι επεμβάσεις θεωρούμε ότι έχουν μόνο έναν κύριο χειρουργό. Συνεπώς, δεν μπορεί να υπάρξει επέμβαση η οποία έχει δύο ή περισσότερους κύριους χειρουργούς. Το `procedure\_event` έχει έναν chief_surgeon_amka. Άλλοι συμμετέχοντες μπαίνουν στον procedure_participant.

3.13. Δεν επιτρέπεται επικάλυψη χώρου ή προσωπικού στις επεμβάσεις. Θεωρούμε ότι ένας χώρος επέμβασης και ένας συμμετέχων δεν μπορούν να χρησιμοποιούνται σε δύο επεμβάσεις που επικαλύπτονται χρονικά.

3.14. Ως σχεδιαστική απλοποίηση, η ιατρική αξιολόγηση αποθηκεύεται σε επίπεδο νοσηλείας και αποδίδεται στους ιατρούς που συμμετείχαν στη νοσηλεία. Αυτό σημαίνει ότι το Q4 υπολογίζει proxy doctor evaluation και όχι ανεξάρτητη αξιολόγηση ανά ιατρό.

4. Relational Schema

Αφού γράψαμε τον κώδικα install.sql χρησιμοποιήσαμε το mysql workbench για να παράγουμε το σχεσιακό σχήμα:



4.1 Η Ακριβής Σειρά των Tables (βάσει του Script):

1. personnel: Η βάση για κάθε εργαζόμενο.
2. doctor: Επέκταση του personnel για γιατρούς.
3. department: Τα τμήματα (συνδέονται με τον Director doctor).
4. nurse: Επέκταση του personnel για νοσηλευτές (συνδέεται με department).
5. administrative_staff: Επέκταση του personnel για διοικητικούς.
6. doctor_department: Ποιοι γιατροί δουλεύουν σε ποια τμήματα (Many-to-Many).
7. bed: Οι κλίνες ανά τμήμα.
8. patient: Τα στοιχεία των ασθενών.
9. emergency_contact: Επαφές ανάγκης για κάθε ασθενή.
10. icd10_diagnosis: Ο κατάλογος με τους κωδικούς ασθενειών.
11. ken: Ο κατάλογος κοστολόγησης νοσηλίων.
12. operating_place: Χώροι χειρουργείων/επεμβάσεων.
13. department_shift: Ορισμός βαρδιών ανά τμήμα και ώρα.
14. shift_assignment: Ποιοι υπάλληλοι δουλεύουν σε ποια βάρδια.
15. emergency_visit: Καταγραφή προσέλευσης στα Επείγοντα (Triage).
16. hospitalization: Το κύριο γεγονός της νοσηλείας (συνδέει τα πάντα).
17. hospitalization_doctor: Όλοι οι γιατροί μιας νοσηλείας.
18. lab_test_catalog: Τι είδους εξετάσεις υπάρχουν.
19. lab_test: Οι συγκεκριμένες εξετάσεις που έγιναν.

20. `procedure_catalog`: Τι είδους επεμβάσεις υπάρχουν.
21. `procedure_event`: Η πραγματοποίηση μιας επέμβασης.
22. `procedure_participant`: Ποιοι συμμετείχαν στην επέμβαση.
23. `drug`: Εμπορικά φάρμακα.
24. `active_substance`: Δραστικές ουσίες.
25. `drug_active_substance`: Ποιες ουσίες έχει κάθε φάρμακο.
26. `patient_allergy`: Αλλεργίες ασθενών σε ουσίες.
27. `prescription`: Οι συνταγογραφήσεις.
28. `hospitalization_evaluation`: Η αξιολόγηση από τον ασθενή.
29. `image_asset`: Τα αρχεία εικόνων/εγγράφων.
30. `entity_image`: Ο σύνδεσμος των εικόνων με οποιοδήποτε table.

Ακεραιότητα: Οι περιορισμοί FOREIGN KEY με ON DELETE RESTRICT σε κρίσιμα δεδομένα (όπως γιατροί ή τμήματα) εμποδίζουν τη διαγραφή στοιχείων που χρησιμοποιούνται σε νοσηλείες.

5. Indexes και Βελτιστοποίηση

Τα ευρετήρια (indexes) αποτελούν δομές δεδομένων που ενσωματώνονται στις σχεσιακές βάσεις δεδομένων με σκοπό τη βελτιστοποίηση της ταχύτητας εκτέλεσης των ερωτημάτων αναζήτησης και ανάκτησης. Επιτρέπουν στο σύστημα διαχείρισης να εντοπίζει άμεσα τις ζητούμενες εγγραφές χωρίς να απαιτείται η χρονοβόρα σάρωση ολόκληρου του πίνακα (full table scan). Στο περιβάλλον της SQL, τα ευρετήρια δημιουργούνται αυτόματα από τη μηχανή της βάσης δεδομένων κάθε φορά που ορίζεται ένα Primary Key ή ένας περιορισμός μοναδικότητας (UNIQUE) σε έναν πίνακα. Στην περίπτωση του PK δημιουργείται ένα clustered index που καθορίζει τη φυσική σειρά αποθήκευσης των δεδομένων στο δίσκο, ενώ για τους περιορισμούς μοναδικότητας δημιουργούνται non-clustered indexes. Αυτές οι αυτόματες δομές, οργανωμένες συνήθως σε μορφή δέντρου (B-Tree), επιτρέπουν στον βελτιστοποιητή ερωτημάτων (query optimizer) να εκτελεί ακαριαία τις λειτουργίες φιλτραρίσματος (WHERE), ταξινόμησης (ORDER BY) και σύνδεσης πινάκων (JOIN), μειώνοντας το υπολογιστικό κόστος και τις διεργασίες εισόδου/εξόδου (I/O) του συστήματος.

Για τα Queries φτιάξαμε τα ακόλουθα indexes τα οποία είδαμε ότι χρησιμοποιήθηκαν με EXPLAIN ANALYZE:

Q1:

idx_hosp_department_admission

Q2:

idx_proc_event_place

idx_shift_assignment_personnel

Q3:

idx_hosp_department_admission

Q4:

Only the primary keys as indexes

Q5:

Full Table Scan χωρίς indexes

Q6:

idx_hosp_patient_admission_q6

Q7:

idx_das_substance

idx_patient_allergy_substance

Q8:

Only the primary keys as indexes

Q9:

idx_hosp_department_admission

Q10:

idx_prescription_hosp_patient_start

Q11:

idx_proc_event_chief

Q12:

idx_shift_assignment_personnel

Q13:

idx_doctor_supervisor

Q14:

idx_hosp_department_admission

Q15:

idx_emergency_visit_level_arrival

6. Triggers

Τα triggers είναι αποθηκευμένες διαδικασίες, οι οποίες ενεργοποιούνται αυτόματα πριν ή μετά από συγκεκριμένες λειτουργίες (INSERT, UPDATE, DELETE). Μπορούν να εκτελεστούν είτε πριν (BEFORE) από μία λειτουργία για την επικύρωση και τον έλεγχο των δεδομένων, είτε μετά (AFTER) για την ενημέρωση άλλων πινάκων. Στην υλοποίησή μας, χρησιμοποιούμε μόνο BEFORE triggers, ώστε να αποτρέπεται η εισαγωγή ή η τροποποίηση δεδομένων που παραβιάζουν τους κανόνες της βάσης μας, πριν αυτά καταγραφούν.

6.1. Εποπτεία Ιατρών

Τα triggers που υλοποιήθηκαν για την εποπτεία των ιατρών είναι trg_doctor_supervision_bi (BEFORE INSERT) και trg_doctor_supervision_bu (BEFORE UPDATE). Οι κανόνες που επιβάλλουν τα συγκεκριμένα triggers είναι:

- Ένας ιατρός δεν μπορεί να είναι επιβλέπων του εαυτού του
- Κάθε ειδικευόμενος πρέπει υποχρεωτικά να έχει επόπτη
- Οι διευθυντές δεν έχουν επόπτη
- Απαγορεύονται οι κυκλικές αλυσίδες εποπτείας

Για τον πρώτο κανόνα, πραγματοποιείται ένας έλεγχος σύγκρισης μεταξύ του AMKA του ιατρού (NEW.amka) και του AMKA του προτεινόμενου επόπτη (NEW.supervisor_amka). Εάν οι δύο τιμές ταυτίζονται, ο trigger διακόπτει τη διαδικασία εμφανίζοντας μήνυμα σφάλματος.

Για τον δεύτερο κανόνα, αρχικά, ελέγχεται η βαθμίδα του ιατρού (NEW.doctor_rank). Εάν ο ιατρός είναι RESIDENT και η στήλη supervisor_amka είναι NULL, το σύστημα απαγορεύει την εισαγωγή και εμφανίζεται μήνυμα σφάλματος, καθώς οι ειδικευόμενοι πρέπει να έχουν επόπτη. Αντίστοιχα, ελέγχουμε και πάλι τη βαθμίδα του ιατρού και εάν είναι DIRECTOR, ο trigger διασφαλίζει ότι η στήλη supervisor_amka παραμένει NULL, δηλαδή απαγορεύει την εισαγωγή επόπτη σε διευθυντή.

Τέλος, για τον έλεγχο και την απαγόρευση κυκλικής εποπτείας, η διαδικασία ξεκινά με τη δήλωση μιας προσωρινής μεταβλητής, `current_supervisor`, η οποία παίρνει αρχικά την τιμή του νέου επόπτη (`NEW.supervisor_amka`). Με αυτόν τον τρόπο, ο έλεγχος ξεκινά από τον άμεσο επόπτη του ιατρού και προχωρά σταδιακά προς τα πάνω στην ιεραρχία. Η διάσχιση της ιεραρχίας γίνεται με τη χρήση ενός `WHILE loop`. Ο βρόχος εκτελείται όσο η μεταβλητή `current_supervisor` δεν είναι `NULL` ή όσο δεν έχει ανιχνευθεί κύκλος. Σε κάθε επανάληψη εκτελείται ένα `SELECT`, ώστε να βρεθεί ο επόπτης του τρέχοντος επόπτη και έτσι να γίνεται μετάβαση στο επόμενο επίπεδο της ιεραρχίας. Αν η αλυσίδα εποπτείας φτάσει σε διευθυντή (`DIRECTOR`), του οποίου ο επόπτης είναι `NULL`, τότε το `loop` τερματίζει, καθώς η ιεραρχία είναι σωστή και δεν υπάρχει κύκλος. Αν σε κάποια επανάληψη η τιμή του `current_supervisor` γίνει ίση με το `NEW.amka` του ιατρού που εισάγεται ή ενημερώνεται, τότε εντοπίζεται κυκλική αλυσίδα εποπτείας. Στην περίπτωση αυτή, εμφανίζεται μήνυμα σφάλματος και σταματά η διαδικασία.

6.2. Ασφαλής Συνταγογράφηση

Τα `triggers` που υλοποιήθηκαν, ώστε να γίνεται σωστά η συνταγογράφηση φαρμάκων είναι `trg_prescription_no_allergy_bi` (`BEFORE INSERT`) και `trg_prescription_no_allergy_bu` (`BEFORE UPDATE`). Τα συγκεκριμένα, απαγορεύουν τη συνταγογράφηση ενός φαρμάκου, όταν ο ασθενής έχει καταγεγραμμένη αλλεργία σε οποιαδήποτε δραστική ουσία του φαρμάκου.

Το `trigger` εκτελεί ένα `EXISTS subquery`, το οποίο ελέγχει αν υπάρχει κοινή δραστική ουσία ανάμεσα στις αλλεργίες του ασθενή και στις ουσίες που περιέχει το φάρμακο το οποίο πρόκειται να συνταγογραφηθεί. Παίρνουμε τις αλλεργίες του ασθενή από τον πίνακα `patient_allergy` χρησιμοποιώντας το `AMKA` του (`NEW.patient_amka`), ενώ τις δραστικές ουσίες που περιλαμβάνει το φάρμακο, τις παίρνουμε από τον πίνακα `drug_active_substance`, χρησιμοποιώντας τον κωδικό του (`NEW.drug_id`). Η ένωση (`JOIN`) των δύο αυτών πινάκων, γίνεται μέσω του `substance_id`, ελέγχοντας αν υπάρχει ουσία του συγκεκριμένου φαρμάκου στην οποία να είναι αλλεργικός ο ασθενής.

Μέσα στο `EXISTS`, χρησιμοποιήσαμε το `SELECT 1`, διότι μας ενδιαφέρει μόνο το αν το `subquery` επιστρέφει τουλάχιστον μία τιμή και όχι το τι επιστρέφεται. Επομένως, αρκεί να επιστρέψουμε μία οποιαδήποτε τιμή, όπως το 1, για να δείξουμε ότι υπήρξε αποτέλεσμα.

Αν ο έλεγχος εντοπίσει έστω και μία ουσία του φαρμάκου στην οποία ο ασθενής είναι αλλεργικός, τότε επιστρέφεται μήνυμα σφάλματος και η διαδικασία της συνταγογράφησης σταματά.

6.3. Αποφυγή Επικάλυψης Χώρων Ιατρικών Πράξεων

Τα `triggers` που υλοποιήθηκαν, ώστε να αποφεύγεται η επικάλυψη χώρων είναι `trg_procedure_room_overlap_bi` (`BEFORE INSERT`) και `trg_procedure_room_overlap_bu` (`BEFORE UPDATE`). Οι κανόνες που επιβάλλουν τα συγκεκριμένα `triggers` είναι:

- Δύο επεμβάσεις δεν μπορούν να πραγματοποιούνται ταυτόχρονα στον ίδιο χώρο

- Ο ίδιος κύριος χειρουργός δεν μπορεί να συμμετέχει σε επικαλυπτόμενες επεμβάσεις

Ο έλεγχος και για τους 2 κανόνες, γίνεται χρησιμοποιώντας τη συνθήκη `NEW.start_ts < pe.end_ts AND NEW.end_ts > pe.start_ts`, διότι δύο χρονικά διαστήματα επικαλύπτονται όταν η νέα επέμβαση πάει να ξεκινήσει πριν την ολοκλήρωση μίας άλλης, καθώς και όταν η νέα επέμβαση ολοκληρώνεται μετά την έναρξη κάποιας από τις υπάρχουσες επεμβάσεις. Έτσι, εντοπίζουμε κάθε πιθανή επικάλυψη που μπορεί να προκύψει. Στο BEFORE UPDATE trigger, προστίθεται και η συνθήκη `pe.procedure_event_id != NEW.procedure_event_id`, ώστε να αποφευχθεί η σύγκριση της εγγραφής με τον εαυτό της.

Όπως και στα παραπάνω triggers, χρησιμοποιούμε EXISTS subquery με SELECT 1. Αν το subquery επιστρέψει κάποια εγγραφή, τότε σημαίνει ότι υπάρχει επικάλυψη και συνεπώς, επιστρέφεται μήνυμα σφάλματος και η εγγραφή ή η ενημέρωση σταματά.

6.4. Συμμετέχοντες σε Ιατρικές Πράξεις

Τα triggers που υλοποιήθηκαν για τη σωστή συμμετοχή του προσωπικού στις ιατρικές πράξεις και για την αποφυγή επικαλύψεων, είναι

<code>trg_procedure_participant_overlap_bi</code>	(BEFORE	INSERT),
<code>trg_procedure_participant_overlap_bu</code>	(BEFORE	UPDATE),
<code>trg_procedure_participant_not_chief_bi</code>	(BEFORE	INSERT),
<code>trg_procedure_participant_not_chief_bu</code>	(BEFORE	UPDATE),
<code>trg_procedure_participant_not_admin_bi</code>	(BEFORE	INSERT),
<code>trg_procedure_participant_not_admin_bu</code>	(BEFORE UPDATE).	

Οι κανόνες που επιβάλλουν τα συγκεκριμένα triggers είναι:

- Δεν μπορεί το ίδιο προσωπικό να συμμετέχει σε επικαλυπτόμενες ιατρικές πράξεις
- Ένας ιατρός δεν μπορεί να είναι κύριος χειρουργός και συμμετέχων σε επικαλυπτόμενες ιατρικές πράξεις
- Ο κύριος χειρουργός δεν καταχωρείται ως συμμετέχων
- Το διοικητικό προσωπικό δεν μπορεί να συμμετέχει σε ιατρικές πράξεις

Για τους 2 πρώτους κανόνες, αρχικά, παίρνουμε από τον πίνακα `procedure_event` την έναρξη(`start_ts`) και τη λήξη (`end_ts`) της ιατρικής πράξης και αποθηκεύουμε τις τιμές αυτές, στις προσωρινές μεταβλητές `v_start` και `v_end`, αντίστοιχα. Στη συνέχεια, χρησιμοποιώντας subquery EXISTS (SELECT 1 ...), ελέγχουμε αν κάποιο μέλος του προσωπικού ή ο κύριος χειρουργός, συμμετέχει σε άλλη ιατρική πράξη, η οποία επικαλύπτεται με την τρέχουσα. Για τον έλεγχο της επικάλυψης, χρησιμοποιούμε, αντίστοιχα με πριν, τη συνθήκη `v_start < pe.end_ts AND v_end > pe.start_ts`. Σε περίπτωση επικάλυψης, εμφανίζεται μήνυμα σφάλματος και η διαδικασία σταματά.

Για τον τρίτο κανόνα, υλοποιήθηκαν τα triggers `trg_procedure_participant_not_chief_bi`, `trg_procedure_participant_not_chief_bu` τα οποία ελέγχουν αν ο συμμετέχων που προσπαθούμε να προσθέσουμε ταυτίζεται με τον κύριο χειρουργό της συγκεκριμένης ιατρικής πράξης. Ο

έλεγχος επιτυγχάνεται, συγκρίνοντας το NEW.personnel_amka με το pe.chief_surgeon_amka. Αν οι τιμές των AMKA είναι οι ίδιες, τότε εμφανίζεται μήνυμα σφάλματος και η εισαγωγή σταματά, καθώς ο κύριος χειρουργός είναι ήδη καταγεγραμμένος στον πίνακα procedure_event. Τέλος, για τον τέταρτο κανόνα, τα triggers trg_procedure_participant_not_admin_bi και trg_procedure_participant_not_admin_bu, ελέγχουν τον τύπο προσωπικού από τον πίνακα personnel. Αν το μέλος προσωπικού ανήκει στην κατηγορία ADMIN, εμφανίζεται μήνυμα σφάλματος και η διαδικασία σταματά, καθώς στις ιατρικές πράξεις δεν επιτρέπεται η συμμετοχή διοικητικού προσωπικού.

6.5. Διασφάλιση Κατάλληλου Χώρου για Ιατρικές Πράξεις

Τα triggers που υλοποιήθηκαν για τη διασφάλιση του κατάλληλου χώρου για την πραγματοποίηση των ιατρικών πράξεων, είναι trg_procedure_place_type_bi (BEFORE INSERT), trg_procedure_place_type_bu (BEFORE UPDATE).

Αρχικά, για να βρούμε τον απαιτούμενο τύπο του χώρου(required_place_type) από τον πίνακα procedure_catalog, χρησιμοποιούμε τον κωδικό της ιατρικής πράξης(NEW.procedure_code). Με αυτόν τον τρόπο, βρίσκουμε για κάθε ιατρική πράξη, αν πρέπει να πραγματοποιηθεί σε χειρουργείο(OPERATING_ROOM) ή σε αίθουσα επέμβασης(PROCEDURE_ROOM) και το αποθηκεύουμε στην προσωρινή μεταβλητή v_required_place_type . Στη συνέχεια, βρίσκουμε τον τύπο του χώρου που πάει να οριστεί για τη συγκεκριμένη ιατρική πράξη(place_type) από τον πίνακα operating_place, μέσω του NEW.place_id και τον αποθηκεύουμε στην προσωρινή μεταβλητή v_actual_place_type. Τέλος, ελέγχουμε αν οι δύο αυτές τιμές ταυτίζονται. Αν δεν ταυτίζονται, τότε εμφανίζεται μήνυμα σφάλματος και η διαδικασία σταματά, καθώς ο επιλεγμένος χώρος δεν είναι αυτός που απαιτείται για την πράξη.

6.6. Όρια Βαρδιών ανά Μήνα

Τα triggers που υλοποιήθηκαν για την επιβολή μηνιαίων ορίων στις βάρδιες, είναι trg_shift_monthly_limits_bi (BEFORE INSERT), trg_shift_monthly_limits_bu(BEFORE UPDATE). Τα μέγιστα μηνιαία όρια που επιβάλλονται είναι:

- 15 βάρδιες για τους ιατρούς
- 20 βάρδιες για τους νοσηλευτές
- 25 βάρδιες για το διοικητικό προσωπικό

Αρχικά, για να βρούμε τον τύπο του μέλους του προσωπικού(personnel_type) από τον πίνακα personnel, χρησιμοποιούμε το AMKA (NEW.personnel_amka). Ανάλογα με τον τύπο του υπαλλήλου, καθορίζουμε το αντίστοιχο μέγιστο όριο και αποθηκεύουμε την τιμή αυτή στην μεταβλητή max_limit. Στη συνέχεια, για να βρούμε την ημερομηνία της νέας βάρδιας(shift_date) από τον πίνακα department_shift, τον συνδέουμε με τον πίνακα shift_assignment, χρησιμοποιώντας το NEW.shift_id και η ημερομηνία αποθηκεύεται στη μεταβλητή new_shift_date. Έπειτα, με το SELECT COUNT(*) υπολογίζονται οι βάρδιες που έχουν ήδη καταχωρηθεί για αυτόν τον μήνα για το κάθε μέλος, μέσω του AMKA τους και αποθηκεύονται στη μεταβλητή shift_cnt. Επίσης, χρησιμοποιήσαμε τις συναρτήσεις MONTH() και YEAR(), για

τον ακριβή διαχωρισμό των βαρδιών ανά μήνα και έτος, διασφαλίζοντας ότι τα όρια ανανεώνονται σωστά την πρώτη ημέρα κάθε μήνα. Τέλος, συγκρίνουμε το shift_cnt με το αντίστοιχο όριο max_limit και αν η ανάθεση βάρδιας που πάει να πραγματοποιηθεί προκαλεί υπέρβαση του επιτρεπτού μηνιαίου ορίου, τότε η διαδικασία σταματά και εμφανίζεται μήνυμα σφάλματος.

6.7. Ελάχιστο Διάστημα Ανάπαυσης

Τα triggers που υλοποιήθηκαν, ώστε να διασφαλιστεί ότι μεταξύ δύο διαδοχικών βαρδιών του ίδιου μέλους πρέπει να μεσολαβούν τουλάχιστον 8 ώρες, είναι trg_shift_rest_bi (BEFORE INSERT), trg_shift_rest_bu (BEFORE UPDATE).

Επειδή οι βάρδιες αποθηκεύονται με ξεχωριστά πεδία ημερομηνίας (shift_date) και ώρας (start_time, end_time), ενώ η νυχτερινή βάρδια ολοκληρώνεται την επόμενη ημέρα από την έναρξή της, στα triggers μετατρέπουμε τις ημερομηνίες και τις ώρες των βαρδιών σε πλήρεις τιμές DATETIME, ώστε να μπορούν να γίνουν σωστοί χρονικοί υπολογισμοί, με χρήση της συνάρτησης TIMESTAMP().

Αρχικά, βρίσκουμε από τον πίνακα department_shift την ημερομηνία και την ώρα έναρξης της νέας βάρδιας που πρόκειται να ανατεθεί ή να ενημερωθεί, μέσω του shift_id και δημιουργούμε ένα πλήρες DATETIME new_start_dt (SET new_start_dt = TIMESTAMP(new_date, new_start)). Στη συνέχεια, βρίσκουμε την αμέσως προηγούμενη βάρδια (TIMESTAMP(ds.shift_date, ds.end_time) <= new_start_dt) του ίδιου ατόμου, συνδέοντας (JOIN) τους πίνακες shift_assignment και department_shift. Για να εξασφαλίσουμε ότι επιλέγεται μόνο η αμέσως προηγούμενη βάρδια, γίνεται φθίνουσα ταξινόμηση των βαρδιών με βάση την ημερομηνία και την ώρα έναρξης, σε συνδυασμό με LIMIT 1 (ORDER BY ds.shift_date DESC, ds.start_time DESC LIMIT 1).

Αφού βρεθεί η προηγούμενη βάρδια, μετατρέπεται η ώρα λήξης της σε πλήρες DATETIME (SET prev_end_dt = TIMESTAMP(old_date, old_end)). Στην περίπτωση βραδινής βάρδιας, η ώρα λήξης της είναι την επόμενη μέρα από την έναρξή της. Επομένως, για να γίνουν σωστά οι υπολογισμοί, προσθέτουμε μία μέρα στο DATETIME της λήξης χρησιμοποιώντας τη συνάρτηση DATE_ADD().

```
IF old_type = 'NIGHT' THEN
```

```
    SET prev_end_dt = DATE_ADD(prev_end_dt, INTERVAL 1 DAY);
```

```
END IF;
```

Τέλος, για τον σωστό και ακριβή υπολογισμό της διαφοράς ωρών ανάμεσα στη λήξη της προηγούμενης βάρδιας και στην έναρξη της νέας, χρησιμοποιήσαμε τη συνάρτηση TIMESTAMPDIFF().

```
IF TIMESTAMPDIFF(HOUR, prev_end_dt, new_start_dt) < 8 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Rest must be at least 8 hours between shifts.';
```

END IF;

Αν η διαφορά είναι μικρότερη από 8 ώρες, τότε εμφανίζεται μήνυμα σφάλματος και η διαδικασία σταματά.

Στο trigger BEFORE UPDATE trigger προστίθεται επιπλέον η συνθήκη sa.shift_id != OLD.shift_id, ώστε η εγγραφή που ενημερώνεται να μη συγκρίνεται με τον εαυτό της κατά την αναζήτηση προηγούμενης βάρδιας.

6.8. Όριο Συνεχόμενων Νυχτερινών Βαρδιών

Τα triggers που υλοποιήθηκαν ώστε να διασφαλιστεί ότι κανένα μέλος του προσωπικού δεν θα αναλάβει περισσότερες από 3 συνεχόμενες νυχτερινές βάρδιες είναι trg_night_shifts_limit_bi (BEFORE INSERT) και trg_night_shifts_limit_bu (BEFORE UPDATE).

Αρχικά, παίρνουμε από τον πίνακα department_shift τον τύπο της νέας βάρδιας (shift_type), χρησιμοποιώντας το NEW.shift_id και συνεχίζουμε περαιτέρω έλεγχο μόνο αν η βάρδια αυτή είναι βραδινή NIGHT. Στην περίπτωση αυτή, χρησιμοποιούμε subquery το οποίο επιστρέφει τις 3 πιο πρόσφατες βάρδιες του εργαζομένου, ταξινομημένες από την πιο πρόσφατη στην πιο παλιά.

```
SELECT ds.shift_type
FROM shift_assignment sa
JOIN department_shift ds ON ds.shift_id = sa.shift_id
WHERE sa.personnel_amka = NEW.personnel_amka
      AND sa.shift_id != NEW.shift_id
ORDER BY ds.shift_date DESC, ds.start_time DESC
LIMIT 3
```

Η ταξινόμηση ORDER BY ds.shift_date DESC, ds.start_time DESC εξασφαλίζει ότι εξετάζονται πρώτα οι πιο πρόσφατες αναθέσεις, ενώ το LIMIT 3 περιορίζει τον έλεγχο μόνο στις τελευταίες τρεις βάρδιες. Το αποτέλεσμα του subquery χρησιμοποιείται ως last_3_shifts και πάνω σε αυτό εφαρμόζεται το εξωτερικό subquery, με το οποίο μετράμε πόσες από τις τελευταίες 3 βάρδιες ήταν νυχτερινές, χρησιμοποιώντας COUNT(*) και αποθηκεύουμε το αποτέλεσμα στη μεταβλητή night_shifts_cnt.

```
SELECT COUNT(*) INTO night_shifts_cnt
FROM (
    SELECT ds.shift_type
    FROM shift_assignment sa
    JOIN department_shift ds ON ds.shift_id = sa.shift_id
    WHERE sa.personnel_amka = NEW.personnel_amka
          AND sa.shift_id != NEW.shift_id
    ORDER BY ds.shift_date DESC, ds.start_time DESC
    LIMIT 3)
```



```
AS last_3_shifts
WHERE shift_type = 'NIGHT';
```

Αν ο αριθμός night_shifts_cnt είναι ίσος ή μεγαλύτερος από 3, τότε με τη νέα νυχτερινή βάρδια υπερβαίνουμε το όριο των συνεχόμενων νυχτερινών βαρδιών και συνεπώς, εμφανίζεται μήνυμα σφάλματος και η διαδικασία ανάθεσης της βάρδιας σταματά.

Στο BEFORE UPDATE trigger χρησιμοποιείται επιπλέον η συνθήκη AND sa.shift_id != NEW.shift_id ώστε η εγγραφή που ενημερώνεται να μη συμπεριληφθεί στον υπολογισμό των προηγούμενων βαρδιών.

6.9. Υπολογισμός Κόστους Νοσηλείας

Το trigger που υλοποιήθηκε για τον υπολογισμό του συνολικού κόστους στο τέλος μίας νοσηλείας, είναι trg_extra_hospitalization_cost_bu (BEFORE UPDATE), το οποίο εκτελείται πριν από κάθε ενημέρωση του πίνακα hospitalization. Η επιλογή του UPDATE αντί του INSERT έγινε, διότι η τελική διάρκεια μίας νοσηλείας μπορεί να υπολογιστεί μόνο όταν ο ασθενής πάρει εξιτήριο και γίνει update στη νοσηλεία του. Αρχικά, χρησιμοποιώντας τον κωδικό ken(ken_code), παίρνουμε από τον πίνακα ken, όσα χρειάζονται για τον υπολογισμό του τελικού κόστους, δηλαδή το βασικό κόστος μίας νοσηλείας(basic_cost), τη μέση διάρκειά της(mean_duration_days) και το επιπλέον κόστος νοσηλείας(extra_daily_cost), σε περίπτωση που η νοσηλεία υπερβαίνει τη μέση διάρκειά της και τα αποθηκεύουμε στις μεταβλητές v_basic_cost, v_mean_duration_days, v_extra_daily_cost, αντίστοιχα. Στη συνέχεια, αφού βρεθούν η ημέρα εισαγωγής(NEW.admission_ts) και η ημέρα που ο ασθενής πήρε εξιτήριο(NEW.discharge_ts), υπολογίζεται η συνολική διάρκεια παραμονής του ασθενούς στο νοσοκομείο μέσω της συνάρτησης DATEDIFF(). Η συνάρτηση επιστρέφει τη διαφορά των δύο ημερομηνιών σε ημέρες και το αποτέλεσμα αποθηκεύεται στη μεταβλητή total_days.

```
SET total_days = DATEDIFF(NEW.discharge_ts, NEW.admission_ts);
```

Αν το total_days είναι μικρότερο ή ίσο από τη μέση διάρκεια νοσηλείας, τότε το συνολικό κόστος είναι ίσο με το βασικό κόστος.

```
IF total_days <= v_mean_duration_days THEN
```

```
    SET NEW.total_cost = v_basic_cost;
```

```
END IF;
```

Αν η νοσηλεία ξεπερνά τη μέση διάρκεια, τότε υπολογίζουμε το πλήθος των επιπλέον ημερών και προσθέτουμε το έξτρα κόστος, με βάση τον τύπο

$$\text{total_cost} = \text{basic_cost} + (\text{total_days} - \text{mean_duration_days}) \times \text{extra_daily_cost}$$

και αποθηκεύουμε την τιμή αυτή στο NEW.total_cost. Έτσι, διασφαλίζουμε τον σωστό υπολογισμό του κόστους μίας νοσηλείας.

6.10. Επικύρωση Βάρδιας

Το trigger που υλοποιήθηκε για τον τελικό έλεγχο μίας βάρδιας πριν οριστικοποιηθεί, είναι `trg_shift_validation_bu` (BEFORE UPDATE), το οποίο εκτελείται πριν από κάθε ενημέρωση στον πίνακα `department_shift`. Κάθε βάρδια βρίσκεται αρχικά σε κατάσταση επεξεργασίας (PROCESSING) μέχρι να πάρει την τελική της μορφή. Όταν μία βάρδια είναι έτοιμη να γίνει VALID, τότε περνάει τους ελέγχους του συγκεκριμένου trigger. Οι κανόνες που επιβάλλονται είναι:

- Κάθε βάρδια πρέπει να αποτελείται από έναν ελάχιστο αριθμό ατόμων κάθε τύπου
- Αν στη βάρδια συμμετέχει ειδικευόμενος, τότε πρέπει υποχρεωτικά να είναι παρών τουλάχιστον ένας ιατρός βαθμίδας Επιμελητής Α΄ ή Διευθυντής.

Αρχικά, ελέγχουμε αν υπάρχει πραγματική αλλαγή κατάστασης προς VALID:

```
IF NEW.shift_status = 'VALID'  
AND OLD.shift_status != 'VALID' THEN
```

Η δεύτερη συνθήκη (`OLD.shift_status != 'VALID'`) είναι σημαντική, επειδή αποτρέπει την επανεκτέλεση των ελέγχων σε ενημερώσεις μιας ήδη επικυρωμένης βάρδιας. Αν υπάρχει πράγματι αλλαγή κατάστασης, τότε καλούνται δύο stored procedures για την εκτέλεση των απαραίτητων ελέγχων(η αναλυτική εξήγηση της υλοποίησης των stored procedures θα γίνει παρακάτω). Αν κάποιο από τα stored procedures εντοπίσει παραβίαση κάποιου κανόνα, τότε εμφανίζεται μήνυμα σφάλματος και δεν πραγματοποιείται η επικύρωση της βάρδιας.

6.11. Διαθεσιμότητα και Δέσμευση Κλινών

Τα triggers που υλοποιήθηκαν για τον έλεγχο διαθεσιμότητας μίας κλίνης και για τη σωστή ενημέρωση της κατάστασης της, είναι triggers `trg_bed_status_bi` (BEFORE INSERT) και `trg_bed_status_bu` (BEFORE UPDATE) και εκτελούνται πριν από κάθε εισαγωγή ή ενημέρωση στον πίνακα `hospitalization`. Αρχικά, κατά την εισαγωγή νέας νοσηλείας, βρίσκουμε την τρέχουσα κατάσταση της κλίνης(`bed_status`) από τον πίνακα `bed` και την αποθηκεύουμε στην μεταβλητή `v_bed_status`. Στη συνέχεια, ελέγχουμε αν η κλίνη δεν είναι διαθέσιμη και στην περίπτωση αυτή, εμφανίζεται μήνυμα σφάλματος και η διαδικασία σταματά.

Στην περίπτωση που η κλίνη είναι διαθέσιμη, γίνεται update στον πίνακα `bed` και πλέον η κατάσταση της κλίνης από AVAILABLE γίνεται OCCUPIED.

```
UPDATE bed  
SET bed_status = 'OCCUPIED'  
WHERE bed_id = NEW.bed_id;
```

Το BEFORE UPDATE trigger, εκτελεί ελέγχους μόνο όταν γίνεται ενημέρωση κάποιας νοσηλείας κατά την οποία αλλάζει η κλίνη(`IF NEW.bed_id != OLD.bed_id`) ή όταν κάποιος ασθενής πάρει εξιτήριο. Όπως και πριν, αρχικά, ελέγχουμε αν η νέα κλίνη είναι διαθέσιμη. Αν δεν είναι, εμφανίζεται μήνυμα σφάλματος και η αλλαγή κλίνης σταματά. Στην περίπτωση που είναι ελεύθερη, πραγματοποιούνται 2 ενημερώσεις. Μία για τη δέσμευση της νέας κλίνης και μία για την αποδέσμευση της προηγούμενης.

```
UPDATE bed
SET bed_status = 'OCCUPIED'
WHERE bed_id = NEW.bed_id;
```

και

```
UPDATE bed
SET bed_status = 'AVAILABLE'
WHERE bed_id = OLD.bed_id;
```

Τέλος, στην περίπτωση εξιτηρίου, όπου δεν υπάρχει μεταφορά σε άλλη κλίνη αλλά η νοσηλεία ολοκληρώνεται (δηλαδή το `discharge_ts` από NULL παίρνει τιμή), αποδεσμεύεται η κλίνη, επαναφέρεται δηλαδή σε κατάσταση AVAILABLE.

```
UPDATE bed
SET bed_status = 'AVAILABLE'
WHERE bed_id = OLD.bed_id;
```

Για την υποστήριξη της μαζικής εισαγωγής δεδομένων για τις νοσηλείες, υλοποιήθηκε μηχανισμός προσωρινής παράκαμψης των `triggers` μέσω της μεταβλητής `@bulk_loading`. Συγκεκριμένα, προστέθηκε : `IF COALESCE(@bulk_loading, 0) = 0`

Η συνθήκη αυτή επιτρέπει στους ελέγχους διαθεσιμότητας κλινών να εκτελούνται μόνο κατά την κανονική λειτουργία του συστήματος, ενώ όταν η μεταβλητή `@bulk_loading` πάρει την τιμή 1, οι `triggers` παρακάμπτουν προσωρινά τους ελέγχους και τις αυτόματες ενημερώσεις κατάστασης των κλινών. Ο μηχανισμός αυτός είναι απαραίτητος για την επιτυχή εισαγωγή των νοσηλειών, καθώς κατά τη μαζική φόρτωση, οι ίδιες κλίνες επαναχρησιμοποιούνται σε διαφορετικές χρονικές περιόδους. Χωρίς την παράκαμψη των `triggers`, οι έλεγχοι διαθεσιμότητας θα θεωρούσαν λανθασμένα ότι οι κλίνες είναι ήδη κατειλημμένες, αποτρέποντας την εισαγωγή δεδομένων.

7. Stored Procedures

Τα Stored Procedures είναι σύνολα εντολών SQL που αποθηκεύονται μόνιμα στη βάση δεδομένων και εκτελούνται ως μια ενιαία διαδικασία. Όπως και τα `triggers`, βοηθούν στην αυτοματοποίηση επαναλαμβανόμενων λειτουργιών, ωστόσο, δεν ενεργοποιούνται αυτόματα από κάποιο συμβάν αλλά καλούνται με την εντολή `CALL`.

7.1. Ελάχιστος Αριθμός Ατόμων μίας Βάρδιας

Το stored procedure που υλοποιήθηκε, ώστε να διασφαλιστεί ότι καμία βάρδια δεν θα επικυρωθεί αν δεν έχει τον ελάχιστο απαραίτητο αριθμό ατόμων κάθε τύπου προσωπικού, είναι shift_composition. Συγκεκριμένα, κάθε βάρδια πρέπει να περιλαμβάνει τουλάχιστον 3 ιατρούς, 6 νοσηλευτές και 2 διοικητικούς υπαλλήλους.

Το procedure δέχεται ως είσοδο το ID της βάρδιας (shiftID), προκειμένου να εξεταστεί το προσωπικό της συγκεκριμένης βάρδιας. Στη συνέχεια, από τον πίνακα shift_assignment, μετράμε το πλήθος του κάθε τύπου προσωπικού που συμμετέχει στη βάρδια, με χρήση COUNT(*).

```
SELECT COUNT(*) INTO doc_cnt
FROM shift_assignment sa
JOIN personnel p ON p.amka = sa.personnel_amka
WHERE sa.shift_id = shiftID
      AND p.personnel_type = 'DOCTOR';
(αντίστοιχα για NURSE, ADMIN)
```

Γίνεται JOIN με τον πίνακα personnel ώστε να ανακτηθεί ο τύπος κάθε εργαζομένου που συμμετέχει στη βάρδια μέσω του πεδίου personnel_type. Με βάση αυτό, πραγματοποιείται ξεχωριστή καταμέτρηση για κάθε κατηγορία προσωπικού μέσα στη συγκεκριμένη βάρδια και τα αποτελέσματα αποθηκεύονται στις μεταβλητές doc_cnt, nurse_cnt και admin_cnt, οι οποίες αντιστοιχούν σε ιατρούς, νοσηλευτές και διοικητικό προσωπικό αντίστοιχα. Τέλος, γίνεται έλεγχος των τιμών αυτών ως προς τα ελάχιστα απαιτούμενα όρια που έχουν οριστεί. Αν οποιαδήποτε από τις τρεις κατηγορίες είναι κάτω από το ελάχιστο όριο, τότε εμφανίζεται μήνυμα σφάλματος και η διαδικασία σταματά, αποτρέποντας την επικύρωση της βάρδιας.

7.2. Επίβλεψη Ειδικευόμενων

Το stored procedure που υλοποιήθηκε, ώστε να ελέγχει αν σε μία βάρδια που συμμετέχει ειδικευόμενος ιατρός, υπάρχει τουλάχιστον ένας Επιμελητής Α ή Διευθυντής, είναι shift_resident_supervisor. Το procedure δέχεται ως είσοδο το ID(shiftID) της βάρδιας, προκειμένου να βρούμε αν υπάρχουν ειδικευόμενοι και Διευθυντές ή Επιμελητές Α.

Αρχικά, υπολογίζουμε πόσοι ειδικευόμενοι υπάρχουν στη βάρδια και στη συνέχεια πόσοι Διευθυντές ή Επιμελητές Α και αποθηκεύουμε τα αποτελέσματα στις μεταβλητές has_resident και has_supervisor αντίστοιχα. Έτσι, βρίσκουμε αν υπάρχει έστω κι ένας ειδικευόμενος.

```
SELECT COUNT(*) INTO has_resident
FROM shift_assignment sa
JOIN doctor d ON d.amka = sa.personnel_amka
```

```
WHERE sa.shift_id = shiftID  
AND d.doctor_rank = 'RESIDENT';
```

```
SELECT COUNT(*) INTO has_supervisor  
FROM shift_assignment sa  
JOIN doctor d ON d.amka = sa.personnel_amka  
WHERE sa.shift_id = shiftID  
AND d.doctor_rank IN ('CONSULTANT_A', 'DIRECTOR');
```

Κάνουμε JOIN τους πίνακες shift_assignment και doctor, ώστε να βρούμε τη βαθμίδα του κάθε συμμετέχοντα ιατρού. Η καταμέτρηση γίνεται με το COUNT(*).

Τέλος, ελέγχουμε αν υπάρχει τουλάχιστον ένας ειδικευόμενος στη βάρδια, χωρίς να υπάρχει κάποιος Διευθυντής ή Επιμελητής A (IF has_resident > 0 AND has_supervisor = 0). Στην περίπτωση αυτή, εμφανίζεται μήνυμα σφάλματος και η διαδικασία σταματά, απορρίπτοντας την επικύρωση της βάρδιας.

7.3. FIFO

Το stored procedure που υλοποιήθηκε για τη διαχείριση της ουράς αναμονής στο Τμήμα Επειγόντων Περιστατικών, είναι FIFO. Το procedure επιστρέφει σαν έξοδο το AMKA του επόμενου ασθενή που θα εξυπηρετηθεί. Η επιλογή του επόμενου ασθενή γίνεται με βάση το επίπεδο επείγοντος και, για το ίδιο επίπεδο, ακολουθείται αυστηρή σειρά προτεραιότητας βάσει χρόνου άφιξης (FIFO). Για την υλοποίηση, αρχικά παίρνουμε από τον πίνακα emergency_visit την κατάσταση του ασθενή (status) για να δούμε αν είναι σε αναμονή ή αν έχει εξυπηρετηθεί και ταξινομούμε τους ασθενείς κατά αύξουσα σειρά επιπέδου επείγοντος και χρόνου άφιξης και με το LIMIT 1 εξασφαλίζουμε ότι επιστρέφεται μία μόνο εγγραφή.

```
SELECT patient_amka, visit_id INTO patientAMKA, v_visit_id  
FROM emergency_visit em  
WHERE em.status = 'WAITING'  
ORDER BY em.emergency_level ASC, em.arrival_ts ASC  
LIMIT 1;
```

Τέλος, μετά την επιλογή του ασθενή, εξασφαλίζουμε ότι δεν θα επιλεγεί ξανά ο ίδιος, αλλάζοντας το status του από WAITING σε CALLED.

```
IF patientAMKA IS NOT NULL THEN  
UPDATE emergency_visit  
SET status = 'CALLED'  
WHERE visit_id = v_visit_id;
```

END IF;

8. Views

Στην υλοποίηση της βάσης δεδομένων χρησιμοποιούνται views, δηλαδή εικονικοί πίνακες που ορίζονται μέσω ενός SELECT ερωτήματος και παρουσιάζουν δεδομένα τα οποία προέρχονται από έναν ή περισσότερους πραγματικούς πίνακες της βάσης. Τα views δεν αποθηκεύουν ανεξάρτητα τα δεδομένα ούτε δημιουργούν διπλότυπα των εγγραφών, αλλά λειτουργούν ως ένα επίπεδο αφαίρεσης πάνω από το κανονικοποιημένο σχεσιακό σχήμα. Η χρήση τους είναι ιδιαίτερα σημαντική στη συγκεκριμένη εργασία, επειδή η βάση δεδομένων του νοσοκομείου περιλαμβάνει πολλές οντότητες που συνδέονται μεταξύ τους, όπως ασθενείς, νοσηλείες, τμήματα, ιατροί, βάρδιες, συνταγογραφήσεις, φάρμακα, δραστικές ουσίες, ιατρικές πράξεις και διαγνώσεις. Αν όλα τα τελικά ερωτήματα έπρεπε να εκτελούν κάθε φορά τα ίδια σύνθετα joins πάνω στους αρχικούς πίνακες, ο SQL κώδικας θα ήταν μεγαλύτερος, λιγότερο ευανάγνωστος και πιο επιρρεπής σε λάθη. Με τα views, η επαναλαμβανόμενη λογική των joins συγκεντρώνεται σε συγκεκριμένα σημεία του σχήματος και τα τελικά ερωτήματα μπορούν να εκφραστούν πιο καθαρά, χρησιμοποιώντας ήδη επεξεργασμένες και νοηματικά ολοκληρωμένες μορφές των δεδομένων. Παράλληλα, τα views δεν αναιρούν την ανάγκη ύπαρξης indexes, καθώς η απόδοση εξακολουθεί να εξαρτάται κυρίως από τα indexes των υποκείμενων πινάκων. Ωστόσο, βελτιώνουν σημαντικά τη συντηρησιμότητα, την αναγνωσιμότητα και την οργάνωση των ερωτημάτων.

8.1.patient_history:

Το σημαντικότερο view για το ιστορικό νοσηλειών είναι το patient_history. Το view αυτό λειτουργεί ως συγκεντρωτική και αναγνώσιμη αναπαράσταση της σχέσης ανάμεσα στον ασθενή και τις νοσηλείες του. Στο κανονικοποιημένο σχήμα, οι πληροφορίες αυτές δεν βρίσκονται σε έναν μόνο πίνακα. Τα δημογραφικά στοιχεία και ο ασφαλιστικός φορέας του ασθενούς βρίσκονται στον πίνακα patient, τα στοιχεία της νοσηλείας στον πίνακα hospitalization, το τμήμα εισαγωγής στον πίνακα department, οι διαγνώσεις στους πίνακες που σχετίζονται με το ICD-10, ενώ η κοστολόγηση της νοσηλείας συνδέεται με το σύστημα KEN. Το patient_history ενοποιεί αυτές τις πληροφορίες σε μία πιο εύχρηστη μορφή, ώστε κάθε γραμμή του view να περιγράφει μία νοσηλεία μαζί με το αντίστοιχο ιατρικό, διοικητικό και οικονομικό πλαίσιο. Έτσι, ο χρήστης του view μπορεί να δει άμεσα το όνομα του ασθενούς, τον ΑΜΚΑ, τον ασφαλιστικό φορέα, το τμήμα στο οποίο νοσηλεύτηκε, την ημερομηνία εισαγωγής και εξόδου, τη διάγνωση, τον κωδικό KEN και το συνολικό κόστος της νοσηλείας, χωρίς να χρειάζεται να γνωρίζει όλες τις εσωτερικές σχέσεις του σχήματος. Επιπλέον, επειδή περιλαμβάνει αναγνώσιμα πεδία, όπως το πλήρες όνομα του ασθενούς και την περιγραφή του τμήματος ή της διάγνωσης,

κάνει τα αποτελέσματα των queries πιο κατανοητά τόσο στο τελικό report όσο και κατά την επίδειξη της βάσης.

8.2.shift_staff:

Το shift_staff αφορά τη λειτουργική οργάνωση των βάρδιών του νοσοκομείου. Η πληροφορία για τις εφημερίες είναι κατανεμημένη σε διαφορετικούς πίνακες, καθώς άλλο πράγμα είναι η ύπαρξη μιας βάρδιας για ένα συγκεκριμένο τμήμα και ημερομηνία, και άλλο πράγμα είναι η ανάθεση συγκεκριμένων μελών προσωπικού σε αυτή τη βάρδια. Το view αυτό ενοποιεί τις πληροφορίες από τους πίνακες που περιγράφουν τις βάρδιες, τα τμήματα, τις αναθέσεις προσωπικού και τα στοιχεία του προσωπικού. Έτσι, παράγει μια πιο ευανάγνωστη εικόνα του προγράμματος εφημεριών, εμφανίζοντας στοιχεία όπως το τμήμα, την ημερομηνία, τον τύπο της βάρδιας, την ώρα έναρξης και λήξης, την κατάσταση της βάρδιας, το ΑΜΚΑ του μέλους προσωπικού, το ονοματεπώνυμο, τον τύπο προσωπικού και τον ρόλο του στη συγκεκριμένη βάρδια. Το συγκεκριμένο view είναι χρήσιμο επειδή η εκφώνηση θέτει αρκετούς περιορισμούς και ερωτήματα σχετικά με τη στελέχωση των βάρδιών, όπως η κάλυψη κάθε τμήματος με συγκεκριμένο αριθμό ιατρών, νοσηλευτών και διοικητικών υπαλλήλων, καθώς και η εύρεση προσωπικού που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα. Αντί κάθε τέτοιο query να επαναλαμβάνει joins μεταξύ department_shift, shift_assignment, department και personnel, μπορεί να χρησιμοποιεί το shift_staff ως ενιαία πηγή πληροφορίας. Επιπλέον, επειδή το view προσθέτει πεδία όπως το shift_year και το πλήρες personnel_name, γίνεται πιο κατάλληλο για grouping, filtering και παρουσίαση αποτελεσμάτων.

8.3. doctor_procedure:

Το view doctor_procedure δημιουργήθηκε για τη συγκεντρωτική και πιο εύχρηστη παρουσίαση πληροφοριών σχετικά με τις ιατρικές πράξεις και τους γιατρούς που τις εκτελούν. Στο κανονικοποιημένο σχήμα, τα δεδομένα αυτά είναι κατανεμημένα σε διαφορετικούς πίνακες: τα στοιχεία της πράξης αποθηκεύονται στο procedure_event, οι πληροφορίες για το είδος της πράξης στο procedure_catalog, τα στοιχεία του γιατρού στους πίνακες doctor και personnel, ενώ ο χώρος διεξαγωγής στο operating_place. Το view περιλαμβάνει τον κωδικό της πράξης, το όνομα και την κατηγορία της, τον χώρο όπου πραγματοποιήθηκε, καθώς και στοιχεία του υπεύθυνου γιατρού, όπως ΑΜΚΑ, ονοματεπώνυμο, ειδικότητα και βαθμίδα. Επιπλέον, περιλαμβάνει χρονικά στοιχεία, όπως την ώρα έναρξης και λήξης της διαδικασίας και την πραγματική διάρκειά της. Με αυτόν τον τρόπο, κάθε γραμμή του doctor_procedure αντιστοιχεί σε μία ολοκληρωμένη καταγραφή ιατρικής πράξης μαζί με τα βασικά ιατρικά στοιχεία της.

8.4. v_doctor_procedure_event :

Το v_doctor_procedure_event χρησιμοποιείται για την ανάλυση των ιατρικών πράξεων και ιδιαίτερα των επεμβάσεων που εκτελούνται από ιατρούς ως κύριοι χειρουργοί. Στο σχήμα της βάσης γίνεται σωστός διαχωρισμός ανάμεσα στον κατάλογο των ιατρικών πράξεων και στα

πραγματικά γεγονότα εκτέλεσης αυτών των πράξεων. Ο πίνακας `procedure_catalog` περιγράφει την επίσημη πράξη, δηλαδή τον κωδικό, το όνομα, την κατηγορία, τη διάρκεια, το κόστος και τον απαιτούμενο χώρο, ενώ ο πίνακας `procedure_event` περιγράφει μία πραγματική εκτέλεση της πράξης κατά τη διάρκεια συγκεκριμένης νοσηλείας. Το `v_doctor_procedure_event` συνδυάζει αυτά τα δεδομένα με τις πληροφορίες του ιατρού, του προσωπικού και του χώρου εκτέλεσης. Έτσι, επιτρέπει την άμεση ανάκτηση στοιχείων όπως το όνομα του ιατρού, η ηλικία του, η ειδικότητα, η βαθμίδα, η πράξη που εκτέλεσε, η κατηγορία της πράξης, το έτος εκτέλεσης, ο χώρος και η διάρκεια. Ιδιαίτερα χρήσιμο είναι ότι το view περιλαμβάνει και πληροφορία για τα τμήματα στα οποία ανήκει ο ιατρός, κάτι που απαιτεί πρόσθετη λογική επειδή ένας ιατρός μπορεί να ανήκει σε περισσότερα από ένα τμήματα. Με αυτόν τον τρόπο, το view μετατρέπει μια αρκετά σύνθετη περιοχή του σχήματος σε ένα καθαρό και αναλυτικό σύνολο δεδομένων. Είναι ιδιαίτερα χρήσιμο για ερωτήματα που ζητούν τους ιατρούς συγκεκριμένης ειδικότητας και τον αριθμό των επεμβάσεων που εκτέλεσαν, τους νέους ιατρούς με τις περισσότερες χειρουργικές επεμβάσεις, καθώς και τους ιατρούς που έχουν εκτελέσει συγκεκριμένο αριθμό επεμβάσεων σε σχέση με τον ιατρό με τη μεγαλύτερη δραστηριότητα. Αν δεν υπήρχε αυτό το view, κάθε τέτοιο query θα έπρεπε να επαναλαμβάνει σύνθετα joins μεταξύ των `procedure`, `doctor`, `personnel`, `department` και `place` tables.

8.5.prescription_substances:

Το `prescription_substances` καλύπτει το τμήμα της φαρμακευτικής αγωγής και των δραστικών ουσιών. Η εκφώνηση δίνει ιδιαίτερη σημασία στο γεγονός ότι οι ασθενείς μπορεί να έχουν αλλεργίες σε δραστικές ουσίες και ότι δεν πρέπει να συνταγογραφείται φάρμακο όταν κάποια από τις δραστικές ουσίες του βρίσκεται στις αλλεργίες του ασθενούς. Για να υποστηριχθεί αυτό το σενάριο, το σχήμα δεν αντιμετωπίζει το φάρμακο ως μία απλή οντότητα χωρίς εσωτερική δομή, αλλά το συνδέει με τις δραστικές ουσίες που περιέχει. Το `prescription_substances` ενώνει τις συνταγογραφήσεις με τα φάρμακα και τις αντίστοιχες δραστικές ουσίες, δημιουργώντας μία ουσιοκεντρική αναπαράσταση της φαρμακευτικής αγωγής. Κάθε εγγραφή του view δείχνει ποιος ασθενής, σε ποια νοσηλεία, έλαβε ποιο φάρμακο, από ποιον ιατρό, σε ποιο χρονικό διάστημα και με ποια δραστική ουσία. Αυτό είναι απαραίτητο για ερωτήματα που δεν ενδιαφέρονται μόνο για το εμπορικό ή γενικό όνομα του φαρμάκου, αλλά για το πραγματικό φαρμακολογικό περιεχόμενό του. Το view βοηθάει άμεσα στην ανάλυση των αλλεργιών ανά δραστική ουσία, στον υπολογισμό του αριθμού φαρμάκων που περιέχουν κάθε ουσία, καθώς και στην εύρεση συχνών ζευγών δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία. Με άλλα λόγια, το `prescription_substances` μετασχηματίζει το υποσύστημα των συνταγογραφήσεων από επίπεδο φαρμάκου σε επίπεδο δραστικής ουσίας, που είναι το κατάλληλο επίπεδο ανάλυσης για τους περιορισμούς ασφαλείας και τα φαρμακευτικά ερωτήματα της εργασίας.

Συμπερασματικά:

Συνολικά, τα views της βάσης λειτουργούν ως ένα ενδιάμεσο επίπεδο μεταξύ του κανονικοποιημένου σχεσιακού μοντέλου και των τελικών αναλυτικών ερωτημάτων. Η βάση παραμένει σωστά δομημένη, με ξεχωριστούς πίνακες, πρωτεύοντα και ξένα κλειδιά, περιορισμούς ακεραιότητας, indexes και triggers, ενώ τα views προσφέρουν μια πιο πρακτική και ευανάγνωστη πρόσβαση στα δεδομένα. Η χρήση τους μειώνει την επανάληψη SQL κώδικα, περιορίζει την πιθανότητα λαθών στα joins και κάνει τα τελικά query files πιο καθαρά. Επιπλέον, τα views βοηθούν στην παρουσίαση των αποτελεσμάτων, επειδή εμφανίζουν πιο κατανοητά πεδία, όπως πλήρη ονόματα ασθενών και προσωπικού, ονόματα τμημάτων, περιγραφές διαγνώσεων, έτη βαρδιών ή επεμβάσεων και ονόματα δραστικών ουσιών. Επομένως, τα views δεν αλλάζουν τη λογική ή τα δεδομένα της βάσης· οργανώνουν όμως την πρόσβαση σε αυτά με τρόπο που εξυπηρετεί καλύτερα τα ζητούμενα ερωτήματα, την τεκμηρίωση και την τελική παρουσίαση της εργασίας.

9. Data generation And Loading

Για τη δημιουργία και τη φόρτωση των δεδομένων της βάσης χρησιμοποιήθηκε μία οργανωμένη διαδικασία δύο σταδίων. Στο πρώτο στάδιο εκτελείται το αρχείο `generate_data.py`, το οποίο παράγει τα απαραίτητα CSV αρχεία για το σχήμα του νοσοκομείου, ενώ στο δεύτερο στάδιο εκτελείται το `load.sql`, το οποίο φορτώνει τα CSV αρχεία στους αντίστοιχους πίνακες της MySQL βάσης. Η λογική αυτή επιλέχθηκε ώστε η παραγωγή των δεδομένων να είναι επαναλήψιμη, ελεγχόμενη και συμβατή με τους περιορισμούς του σχήματος. Το script παραγωγής χρησιμοποιεί σταθερό seed, άρα τα παραγόμενα δεδομένα είναι deterministic και μπορούν να αναπαραχθούν με τον ίδιο τρόπο σε διαφορετικό υπολογιστή ή σε νέα εγκατάσταση της βάσης. Παράλληλα, ο generator δεν δημιουργεί απλώς τυχαία δεδομένα, αλλά query-driven δεδομένα, δηλαδή δεδομένα σχεδιασμένα ώστε να καλύπτουν τα ζητούμενα ερωτήματα της εργασίας, τους περιορισμούς ακεραιότητας, τα triggers, τα views και τις τελικές αναφορές. Αυτό είναι σημαντικό, γιατί η εκφώνηση απαιτεί όλα τα ερωτήματα να επιστρέφουν έγκυρα αποτελέσματα και όχι άδειους πίνακες. Το ίδιο το `generate_data.py` περιγράφει τον ρόλο του ως παραγωγή deterministic CSV δεδομένων για το σχήμα του νοσοκομείου, συνδυάζοντας καθαρισμένα επίσημα reference αρχεία με συνθετικά transactional δεδομένα που ελέγχουν τους περιορισμούς, τα triggers, τα views και τα τελικά reporting queries.

Η διαδικασία παραγωγής δεδομένων χωρίζει τα δεδομένα σε δύο βασικές κατηγορίες: reference δεδομένα και generated/synthetic δεδομένα. Τα reference δεδομένα αντιστοιχούν σε πίνακες που δεν πρέπει να επινοηθούν αυθαίρετα, όπως οι διαγνώσεις ICD-10, τα KEN, ο κατάλογος ιατρικών πράξεων, οι εργαστηριακές εξετάσεις, τα φάρμακα και οι δραστικές ουσίες. Για αυτά, το script προσπαθεί να χρησιμοποιήσει επίσημες ή ήδη καθαρισμένες πηγές, να τις κανονικοποιήσει στη μορφή που απαιτεί το σχεσιακό σχήμα και να τις αποθηκεύσει στον φάκελο `data/reference`. Για παράδειγμα, η πληροφορία των KEN καθαρίζεται ώστε να περιέχει

κωδικό KEN, περιγραφή, βασικό κόστος, μέση διάρκεια νοσηλείας και πρόσθετο ημερήσιο κόστος. Αντίστοιχα, οι ICD-10 κωδικοί καθαρίζονται και αποθηκεύονται σε μορφή κατάλληλη για άμεση φόρτωση στον πίνακα `icd10_diagnosis`. Για τις ιατρικές πράξεις, το `script` διαβάζει τον επίσημο κατάλογο, κρατά τον κωδικό και την ονομασία της πράξης και παράγει επιπλέον κατηγορία πράξης και απαιτούμενο τύπο χώρου, ώστε τα δεδομένα να είναι συμβατά με τους περιορισμούς των `procedure events`. Στην περίπτωση των φαρμάκων, η παραγωγή βασίζεται στο EMA Article 57 όταν αυτό είναι διαθέσιμο· διαφορετικά, ο `generator` δεν κατασκευάζει ψεύτικα φαρμακευτικά δεδομένα, ώστε να μην εισάγονται ανεπίσημα στοιχεία σε πίνακες που κανονικά βασίζονται σε εξωτερική `reference` πηγή.

Τα `synthetic` δεδομένα παράγονται για τους επιχειρησιακούς πίνακες του νοσοκομείου, δηλαδή για τα τμήματα, το προσωπικό, τους ασθενείς, τις κλίνες, τις βάρδιες, τα επείγοντα περιστατικά, τις νοσηλείες, τις εργαστηριακές εξετάσεις, τις ιατρικές πράξεις, τις συνταγογραφήσεις, τις αλλεργίες, τις αξιολογήσεις και τις εικόνες. Ο `generator` δημιουργεί ένα πλήρες νοσοκομειακό περιβάλλον με 15 τμήματα, ιατρούς διαφορετικών ειδικοτήτων και βαθμίδων, νοσηλευτές, διοικητικό προσωπικό, κλίνες και χώρους επεμβάσεων. Ιδιαίτερη προσοχή δίνεται στην παραγωγή προσωπικού, επειδή το σχήμα έχει αρκετούς περιορισμούς γύρω από τις βάρδιες και την ιεραρχία των ιατρών. Οι ιατροί παράγονται με ειδικότητα, βαθμίδα και `supervisor`, ακολουθώντας μία λογική από πάνω προς τα κάτω, ώστε να αποφεύγονται κύκλοι εποπτείας. Τα τμήματα συνδέονται με διευθυντή ιατρό, οι νοσηλευτές και οι διοικητικοί υπάλληλοι ανήκουν σε τμήματα, ενώ οι ιατροί μπορούν να ανήκουν σε ένα ή περισσότερα τμήματα μέσω του πίνακα `doctor_department`.

Για τις βάρδιες, το `script` παράγει πρόγραμμα εφημεριών για συγκεκριμένη εβδομάδα, καλύπτοντας όλα τα τμήματα και τους τρεις τύπους βάρδιας: πρωινή, απογευματινή και νυχτερινή. Οι εγγραφές των βαρδιών δημιουργούνται αρχικά με κατάσταση `PROCESSING`, ώστε να φορτωθούν πρώτα οι αναθέσεις προσωπικού στον πίνακα `shift_assignment`. Στη συνέχεια, κατά τη φόρτωση, οι βάρδιες μετατρέπονται σε `VALID`, ώστε να ενεργοποιηθούν οι έλεγχοι σύνθεσης βάρδιας, δηλαδή οι κανόνες που απαιτούν επαρκή αριθμό ιατρών, νοσηλευτών και διοικητικών υπαλλήλων. Αυτή η επιλογή είναι πρακτική, γιατί δεν μπορεί να χαρακτηριστεί μία βάρδια ως πλήρως έγκυρη πριν φορτωθούν οι αντίστοιχες εγγραφές προσωπικού που τη στελεχώνουν. Έτσι, η παραγωγή και η φόρτωση των δεδομένων ακολουθούν τη λογική των `foreign keys` και των `triggers`, αντί να προσπαθούν να τους παρακάμψουν.

Για τις νοσηλείες, ο `generator` παράγει εγγραφές που συνδέουν ασθενείς, τμήματα, κλίνες, KEN και ICD-10 διαγνώσεις. Κάθε νοσηλεία έχει ημερομηνία εισαγωγής, ημερομηνία εξόδου, κωδικό εισαγωγής και εξόδου ICD-10, KEN και συνολικό κόστος. Το συνολικό κόστος υπολογίζεται με βάση το βασικό κόστος του KEN και την πιθανή υπέρβαση της μέσης διάρκειας νοσηλείας. Επιπλέον, υπάρχει μηχανισμός `allocation` κλινών, ώστε να αποφεύγονται επικαλύψεις νοσηλείων στην ίδια κλίνη. Παράγονται επίσης σχέσεις νοσηλείας-ιατρών, ώστε κάθε νοσηλεία να έχει τουλάχιστον έναν σχετικό ιατρό, καθώς και αξιολογήσεις για σημαντικό μέρος των

ολοκληρωμένων νοσηλειών. Αυτό βοηθάει ιδιαίτερα στα ερωτήματα που αφορούν το ιστορικό νοσηλειών, το κόστος ανά νοσηλεία, τις αξιολογήσεις ασθενών και τις αναλύσεις ανά τμήμα ή ασφαλιστικό φορέα.

Η παραγωγή των procedure events γίνεται επίσης με ελεγχόμενο τρόπο. Το script δεν δημιουργεί απλώς μία τυχαία πράξη, αλλά επιλέγει πράξεις από τον επίσημο κατάλογο, ελέγχει τον απαιτούμενο τύπο χώρου και αντιστοιχίζει την πράξη σε κατάλληλο χειρουργείο ή αίθουσα επέμβασης. Παράλληλα, κρατά προγράμματα χρήσης χώρων και ιατρών, ώστε να αποφεύγεται η ταυτόχρονη εκτέλεση δύο επεμβάσεων στον ίδιο χώρο ή η ταυτόχρονη συμμετοχή του ίδιου ιατρού σε δύο επεμβάσεις. Αυτό είναι σημαντικό επειδή η εκφώνηση απαιτεί τέτοιους λειτουργικούς περιορισμούς. Οι procedure events συνδέονται με νοσηλείες, κύριο χειρουργό και συμμετέχον προσωπικό, ώστε να μπορούν να απαντηθούν τα ερωτήματα που μετρούν επεμβάσεις ανά ιατρό, βρίσκουν νέους ιατρούς με υψηλή χειρουργική δραστηριότητα ή συγκρίνουν τον αριθμό επεμβάσεων μεταξύ ιατρών.

Η παραγωγή των δεδομένων είναι επίσης σχεδιασμένη ώστε να καλύπτει συγκεκριμένα query patterns. Για παράδειγμα, δημιουργούνται ασθενείς που έχουν περισσότερες από τρεις νοσηλείες στο ίδιο τμήμα, ώστε να επιστρέφει αποτέλεσμα το σχετικό ερώτημα. Δημιουργούνται επίσης ασθενείς με ίση συνολική διάρκεια νοσηλείας μέσα στο ίδιο έτος και διάρκεια πάνω από 15 ημέρες, ώστε να υποστηρίζεται το αντίστοιχο grouping query. Αντίστοιχα, δημιουργούνται ICD-10 κατηγορίες με ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, procedure counts με άνιση κατανομή ανά χειρουργό, και emergency visits με διαφορετικά επίπεδα επείγοντος, χρόνους αναμονής, παραπομπές και διαθέσεις. Ο generator καταγράφει αυτή τη λογική και παράγει βοηθητικό αρχείο QUERY_COVERAGE.csv, στο οποίο περιγράφεται ποιο seed rule εξυπηρετεί κάθε τελικό ερώτημα. Για παράδειγμα, αναφέρει ότι υπάρχουν νέοι ιατροί κάτω των 35 ετών με procedure events, ασθενείς με ιστορικά νοσηλειών και αξιολογήσεις, πλήρες εβδομαδιαίο πρόγραμμα βαρδιών για όλα τα τμήματα και τα τρία shifts, καθώς και emergency visits που καλύπτουν τα επίπεδα 1 έως 5.

Μετά την παραγωγή των CSV αρχείων ακολουθεί στάδιο validation. Το generate_data.py ελέγχει ότι οι τιμές που δημιουργήθηκαν είναι συμβατές με τα reference tables και με τους βασικούς περιορισμούς του σχήματος. Για παράδειγμα, ελέγχεται ότι οι κωδικοί KEN που χρησιμοποιούνται στις νοσηλείες υπάρχουν πράγματι στον πίνακα ken, ότι οι ICD-10 κωδικοί υπάρχουν στον πίνακα icd10_diagnosis, ότι οι κωδικοί εργαστηριακών εξετάσεων υπάρχουν στον αντίστοιχο κατάλογο, ότι οι procedure codes υπάρχουν στον procedure_catalog, και ότι ο χώρος εκτέλεσης μιας πράξης ταιριάζει με τον απαιτούμενο τύπο χώρου της πράξης. Επίσης ελέγχεται η συμβατότητα συνταγογραφήσεων, φαρμάκων, δραστικών ουσιών και αλλεργιών. Αυτός ο έλεγχος είναι κρίσιμος, επειδή εντοπίζει λάθη πριν επιχειρηθεί η φόρτωση στη MySQL, όπου ένα λάθος foreign key ή ένας trigger θα μπορούσε να σταματήσει τη διαδικασία εισαγωγής.

Το δεύτερο στάδιο είναι η φόρτωση των δεδομένων μέσω του load.sql. Το αρχείο αυτό χρησιμοποιεί εντολές LOAD DATA LOCAL INFILE, οι οποίες είναι κατάλληλες για μαζική εισαγωγή CSV δεδομένων σε MySQL/MariaDB. Η φόρτωση γίνεται με ρητές λίστες στηλών, ώστε η σειρά των πεδίων κάθε CSV να αντιστοιχίζεται ξεκάθαρα στις στήλες του αντίστοιχου πίνακα. Επιπλέον, χρησιμοποιείται CHARACTER SET utf8mb4, ώστε να υποστηρίζονται σωστά ελληνικοί χαρακτήρες και ειδικά σύμβολα. Το load.sql ορίζει ότι τα πεδία διαχωρίζονται με κόμμα, μπορούν να είναι μέσα σε εισαγωγικά και ότι η πρώτη γραμμή κάθε CSV αγνοείται, επειδή περιέχει headers. Επίσης, για αρκετά πεδία χρησιμοποιούνται user variables και SET expressions, ώστε να γίνεται καθαρισμός τιμών, μετατροπή κενών strings σε NULL μέσω NULLIF, και αφαίρεση πιθανών χαρακτήρων \r από CSV αρχεία που μπορεί να έχουν δημιουργηθεί με Windows-style line endings. Το αρχείο φόρτωσης αναφέρει ρητά ότι πρέπει να εκτελείται από το repository root, επειδή τα paths των CSV αρχείων είναι relative προς τον φάκελο όπου υπάρχει ο κατάλογος data/.

Η σειρά φόρτωσης στο load.sql δεν είναι τυχαία. Ακολουθεί τις εξαρτήσεις των foreign keys, ώστε να εισάγονται πρώτα οι πίνακες αναφοράς και οι γονικές οντότητες, και στη συνέχεια οι πίνακες που εξαρτώνται από αυτούς. Αρχικά φορτώνονται τα reference tables, όπως icd10_diagnosis, ken, procedure_catalog, lab_test_catalog, drug, active_substance και drug_active_substance. Στη συνέχεια φορτώνεται το προσωπικό, οι ιατροί, τα τμήματα, οι συσχετίσεις ιατρών-τμημάτων, οι νοσηλευτές, το διοικητικό προσωπικό, οι κλίνες, οι χώροι επεμβάσεων και οι ασθενείς. Έπειτα φορτώνονται οι βάρδιες και οι αναθέσεις προσωπικού, και μετά ακολουθούν τα emergency visits, οι νοσηλείες, οι ιατροί ανά νοσηλεία, οι εργαστηριακές εξετάσεις, τα procedure events, οι συμμετέχοντες σε πράξεις, οι αλλεργίες, οι συνταγογραφήσεις, οι αξιολογήσεις και οι εικόνες. Αυτή η σειρά είναι απαραίτητη, γιατί για παράδειγμα δεν μπορεί να φορτωθεί μία νοσηλεία αν δεν υπάρχουν ήδη ο ασθενής, το τμήμα, η κλίνη, το KEN και οι ICD-10 κωδικοί που αναφέρονται σε αυτή. Αντίστοιχα, δεν μπορεί να φορτωθεί μία συνταγογράφηση αν δεν υπάρχουν ήδη ο ασθενής, ο ιατρός, η νοσηλεία και το φάρμακο. Το ίδιο το loader ακολουθεί αυτή τη λογική ξεκινώντας από τα reference tables και στη συνέχεια περνώντας σε personnel, doctor, department, doctor_department, nurse, administrative_staff, bed, operating_place και patient.

Συνολικά, η διαδικασία generation και load έχει σχεδιαστεί ώστε να είναι ασφαλής, επαναλήψιμη και συμβατή με τη λογική του σχεσιακού μοντέλου. Το generate_data.py αναλαμβάνει να παράγει καθαρά και συνεπή CSV αρχεία, διαχωρίζοντας τα επίσημα reference δεδομένα από τα συνθετικά transactional δεδομένα. Το load.sql αναλαμβάνει να τα φορτώσει στη βάση με τη σωστή σειρά, δίνοντας έμφαση στα foreign keys, τα nullable πεδία, το encoding και τους triggers. Με αυτόν τον τρόπο, η βάση δεν γεμίζει με απλό τυχαίο περιεχόμενο, αλλά με δεδομένα που έχουν παραχθεί ώστε να ελέγχουν ουσιαστικά τη λειτουργία του σχήματος και να επιτρέπουν την επιτυχή εκτέλεση των 15 ζητούμενων ερωτημάτων. Η επιλογή αυτή βελτιώνει την αξιοπιστία της εργασίας, επειδή δείχνει ότι η βάση δεν έχει σχεδιαστεί μόνο θεωρητικά,

αλλά μπορεί να φορτωθεί μαζικά, να ελεγχθεί και να χρησιμοποιηθεί για πραγματικά σύνθετα SQL queries.

10. Queries

Ακολουθεί μια επεξήγηση των Queries που αναπτύχθηκαν για τις ερωτήσεις 1-15.

Q1:

Αυτό το Query πραγματοποιεί μια ανάλυση των ολοκληρωμένων νοσηλείων στη βάση δεδομένων. Αρχικά, μέσω του πρώτου προσωρινού πίνακα CTE (`hospitalization_days`), απομονώνει τις νοσηλείες που έχουν ολοκληρωθεί και υπολογίζει την πραγματική διάρκειά τους σε ημέρες, μετατρέποντας τη διαφορά ωρών εισαγωγής και εξιτηρίου με στρογγυλοποίηση προς τα πάνω και εξασφαλίζοντας ένα ελάχιστο όριο μίας ημέρας. Στη συνέχεια, στον δεύτερο πίνακα (`hospitalization_cost_parts`), συνδέει τις νοσηλείες με τον πίνακα των Κλειστών Ενοποιημένων Νοσηλίων (KEN) προκειμένου να εντοπίσει τις ημέρες υπέρβασης σε σχέση με τη θεσμοθετημένη μέση διάρκεια νόσου, διαχωρίζοντας το βασικό κόστος από τις επιπλέον ημερήσιες χρεώσεις λόγω παράτασης της νοσηλείας. Στο τέλος, συνδέει τα παραπάνω στοιχεία με τους πίνακες των τμημάτων και των ασθενών, ομαδοποιώντας τα δεδομένα ανά τμήμα, έτος εισαγωγής, κατηγορία KEN και ασφαλιστικό φορέα. Με αυτόν τον τρόπο παράγει δείκτες, όπως το πλήθος των νοσηλείων, τον μέσο όρο ημερών παραμονής, καθώς και μια σύγκριση ανάμεσα στα θεωρητικά υπολογιζόμενα έσοδα και τα ήδη αποθηκευμένα ποσά για λόγους οικονομικού ελέγχου, ταξινομώντας το τελικό αποτέλεσμα χρονολογικά και με βάση τους πιο κερδοφόρους συνδυασμούς για το νοσοκομείο.

Q2:

Αυτό το Query δημιουργεί μια ολοκληρωμένη ετήσια αναφορά δραστηριότητας για τους γιατρούς μιας συγκεκριμένης ειδικότητας (εν προκειμένω, της Καρδιολογίας) στη βάση δεδομένων `ygeiopolis`, αντλώντας αυτόματα το τρέχον έτος μέσω συναρτήσεων συστήματος. Αρχικά, ορίζει τη κωδικοποίηση χαρακτήρων για την ορθή διαχείριση των ελληνικών και χρησιμοποιεί τρεις προσωρινούς πίνακες (CTEs): ο πρώτος (`shiftent`) καταμετρά τις συνολικές μοναδικές βάρδιες κάθε γιατρού για το έτος, ο δεύτερος (`surgent`) υπολογίζει το πλήθος των επεμβάσεων που κατηγοριοποιούνται ως χειρουργικές και στις οποίες ο γιατρός ήταν επικεφαλής, και ο τρίτος (`target_doctors`) απομονώνει τα βασικά στοιχεία των γιατρών που ανήκουν αποκλειστικά στην επιλεγμένη ειδικότητα. Στο τέλος, το ερώτημα συνδέει τη λίστα των καρδιολόγων με τις μετρήσεις των βαρδιών και των χειρουργείων χρησιμοποιώντας `LEFT JOIN`, γεγονός που διασφαλίζει ότι θα συμπεριληφθούν στην αναφορά όλοι οι γιατροί της ειδικότητας ακόμη και αν έχουν μηδενική δραστηριότητα φέτος. Παράλληλα, με τη χρήση της `COALESCE` μετατρέπει τις κενές τιμές `NULL` σε μηδενικά για καθαρότερη εικόνα, εισάγει μια στήλη ελέγχου ενεργότητας ('YES'/'NO') αναφορικά με τις βάρδιες και ταξινομεί το τελικό αποτέλεσμα με βάση τον αριθμό των χειρουργικών επεμβάσεων, έπειτα με βάση τον αριθμό των βαρδιών και, σε περίπτωση απόλυτης ισοβαθμίας, αλφαβητικά με βάση το όνομα του γιατρού.

Q3:

Αυτό το Query εντοπίζει τους συχνούς ασθενείς του νοσοκομείου στη βάση δεδομένων ygeiopolis, επιστρέφοντας όσους έχουν πραγματοποιήσει περισσότερες από τρεις νοσηλείες (δηλαδή τουλάχιστον τέσσερις) μέσα στο ίδιο συγκεκριμένο τμήμα. Αντλώντας δεδομένα από το υπάρχον view patient_history, το ερώτημα ομαδοποιεί τις εγγραφές ανά ασθενή και ανά κλινική, υπολογίζει το πλήθος των εισαγωγών τους και αθροίζει το συνολικό οικονομικό κόστος αυτών των νοσηλείων στρογγυλοποιημένο σε δύο δεκαδικά ψηφία. Τέλος, μέσω του φίλτρου HAVING (>3), απομονώνει αποκλειστικά τις περιπτώσεις υψηλής συχνότητας και ταξινομεί τα αποτελέσματα σε φθίνουσα σειρά, φέρνοντας στην κορυφή της λίστας τους ασθενείς με τον μεγαλύτερο αριθμό επανεισαγωγών και, σε περίπτωση ισοβαθμίας, εκείνους που επιβάρυναν περισσότερα οικονομικά το εκάστοτε τμήμα.

Q4(EXPLAIN ANALYZE):

Το Q4 υπολογίζει τις αξιολογήσεις για έναν συγκεκριμένο ιατρό, ο οποίος επιλέγεται αρχικά ως ο ιατρός με το μεγαλύτερο πλήθος αξιολογημένων νοσηλείων, ώστε το αποτέλεσμα να είναι μη κενό και αντιπροσωπευτικό. Στη συνέχεια, το query συνδέει τον πίνακα doctor με τον πίνακα personnel για να εμφανίσει τα στοιχεία του ιατρού, τον πίνακα hospitalization_doctor για να εντοπίσει τις νοσηλείες στις οποίες συμμετείχε, και τον πίνακα hospitalization_evaluation για να ανακτήσει τις αξιολογήσεις αυτών των νοσηλείων. Τέλος, με χρήση COUNT(DISTINCT he.hosp_id) υπολογίζεται το πλήθος των αξιολογημένων νοσηλείων, ενώ με AVG υπολογίζονται ο μέσος όρος της ποιότητας ιατρικής φροντίδας και ο μέσος όρος της συνολικής εμπειρίας νοσηλείας, στρογγυλοποιημένοι σε δύο δεκαδικά ψηφία.

```
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelous$ mysql -u root --vertical < sql/Q04.sql
***** 1. row *****
EXPLAIN: -> Group aggregate: avg(he.overall_experience_score), avg(he.medical_care_score), count(distinct he.hosp_id) (cost=69.3 rows=1) (actual time=0.589..0.589 rows=1 loops=1)
-> Nested loop inner join (cost=46 rows=101) (actual time=0.0786..0.533 rows=78 loops=1)
-> Filter: ((hd.doctor_amka = '10000000315') and <cache>(('10000000315' = (@target_doctor_amka)))) (cost=10.6 rows=101) (actual time=0.0618..0.15 rows=101 loops=1)
-> Covering index lookup on hd using fk_hosp_doctor_doctor (doctor_amka = (@target_doctor_amka)) (cost=10.6 rows=101) (actual time=0.0581..0.103 rows=101 loops=1)
-> Single-row index lookup on he using PRIMARY (hosp_id = hd.hosp_id) (cost=0.251 rows=1) (actual time=0.00337..0.00343 rows=0.772 loops=101)
```

Στην έκδοση του Q4 με FORCE INDEX, ο optimizer αναγκάζεται να χρησιμοποιήσει το index fk_hosp_doctor_doctor στον πίνακα hospitalization_doctor. Το πλάνο εκτέλεσης δείχνει ότι η αναζήτηση ξεκινά από τις εγγραφές του συγκεκριμένου ιατρού, μέσω doctor_amka, και επιστρέφει 101 σχετικές νοσηλείες. Στη συνέχεια, για κάθε νοσηλεία πραγματοποιείται single-row index lookup στον πίνακα hospitalization_evaluation μέσω του primary key στο hosp_id. Από τις 101 νοσηλείες, βρέθηκαν τελικά 78 αντίστοιχες αξιολογήσεις, πάνω στις οποίες υπολογίζονται οι μέσοι όροι του medical_care_score και του overall_experience_score. Ο συνολικός πραγματικός χρόνος εκτέλεσης είναι περίπου 0.59 ms, γεγονός που δείχνει ότι το query εκτελείται αποδοτικά, χωρίς full table scan, αξιοποιώντας index-based lookups στα βασικά βήματα του πλάνου.

```

MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q04.sql > sql/Q04_out.txt

real    0m0.150s
user    0m0.026s
sys     0m0.015s
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q04.sql > sql/Q04_out.txt

real    0m0.142s
user    0m0.025s
sys     0m0.016s
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q04.sql > sql/Q04_out.txt

real    0m0.142s
user    0m0.027s
sys     0m0.018s
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q04.sql > sql/Q04_out.txt

real    0m0.133s
user    0m0.026s
sys     0m0.016s
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$

```

Παρατηρούμε επιπλέον, χρησιμοποιώντας το force index στο συγκεκριμένο ερώτημα, λόγω των απαιτήσεων της άσκησης, έχουμε πολύ ικανοποιητικό χρόνο εκτέλεσης, κάτι το οποίο επιβεβαιώνεται και από το EXPLAIN ANALYZE που χρησιμοποιήσαμε.

```

MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ mysql -u root --vertical < sql/Q04.sql
***** 1. row *****
EXPLAIN: -> Group aggregate: avg(he.overall_experience_score), avg(he.medical_care_score), count(distinct he.hosp_id)
(cost=8246 rows=1) (actual time=28.1..28.1 rows=1 loops=1)
-> Nested loop inner join (cost=6204 rows=8862) (actual time=0.386..28.1 rows=78 loops=1)
-> Filter: <cache>(('10000000315' = (@target_doctor_amka))) (cost=3102 rows=8862) (actual time=0.22..3.97 rows=8822 loops=1)
-> Table scan on he (cost=3102 rows=8862) (actual time=0.219..3.02 rows=8822 loops=1)
-> Filter: (hd.doctor_amka = '10000000315') (cost=0.25 rows=1) (actual time=0.00257..0.00257 rows=0.00884 loops=8822)
-> Single-row covering index lookup on hd using PRIMARY (hosp_id = he.hosp_id, doctor_amka = (@target_doctor_amka)) (cost=0.25 rows=1) (actual time=0.00236..0.00236 rows=0.00884 loops=8822)

```

Στη μη βέλτιστη έκδοση του Q4, το EXPLAIN ANALYZE δείχνει ότι ο optimizer ξεκινά την εκτέλεση με πλήρη σάρωση του πίνακα hospitalization_evaluation, διαβάζοντας όλες τις διαθέσιμες αξιολογήσεις. Στη συνέχεια, για κάθε αξιολόγηση πραγματοποιεί αναζήτηση στον πίνακα hospitalization_doctor, ώστε να ελέγξει αν η συγκεκριμένη νοσηλεία αντιστοιχεί στον επιλεγμένο ιατρό. Παρόλο που τελικά επιστρέφονται μόνο 78 σχετικές αξιολογήσεις, το query εξετάζει περίπου 8822 εγγραφές και εκτελεί αντίστοιχο αριθμό επαναλήψεων στο join. Αυτό οδηγεί σε πραγματικό χρόνο εκτέλεσης περίπου 28.1 ms, σημαντικά μεγαλύτερο από την πιο αποδοτική έκδοση, όπου η αναζήτηση ξεκινά απευθείας από τις νοσηλείες του συγκεκριμένου ιατρού. Συνεπώς, η διαφορά των δύο πλάνων δείχνει ότι η σειρά εκτέλεσης και η έγκαιρη χρήση του φίλτρου doctor_amka επηρεάζουν σημαντικά την απόδοση του ερωτήματος.

Όσο αφορά το runtime, αν και οι μικρές διαφορές ανάμεσα στις δύο εκδοχές αποτυπώνονται στο EXPLAIN ANALYZE section, δεν μπορούμε να παρατηρήσουμε κάποια δραστική αλλαγή.

Q5:

Το ερώτημα αυτό εντοπίζει τους νέους ιατρούς ηλικίας κάτω των 35 ετών που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί. Αρχικά, το CTE young_surgeon_counts χρησιμοποιεί το view v_doctor_procedure_event, φιλτράρει μόνο τους ιατρούς με doctor_age < 35 και μόνο τις πράξεις κατηγορίας SURGICAL, και στη συνέχεια ομαδοποιεί ανά ιατρό μετρώντας τα διαφορετικά procedure_event_id. Το HAVING COUNT(...) > 0 κρατά μόνο όσους έχουν πράγματι συμμετοχή ως chief σε τουλάχιστον μία χειρουργική

επέμβαση. Έπειτα, το δεύτερο CTE max_count βρίσκει τη μέγιστη τιμή επεμβάσεων ανάμεσα στους νέους ιατρούς. Τέλος, το βασικό SELECT επιστρέφει μόνο τους ιατρούς των οποίων το πλήθος επεμβάσεων ισούται με αυτή τη μέγιστη τιμή, ώστε να καλύπτονται και περιπτώσεις ισοβαθμίας.

Q6(EXPLAIN-ANALYZE):

Το Q06 σχεδιάστηκε ώστε να επιστρέφει το πλήρες ιστορικό νοσηλειών ενός συγκεκριμένου ασθενή. Αρχικά επιλέγεται ένας ασθενής που διαθέτει αξιολογημένες νοσηλείες, ώστε το query να επιστρέφει πλήρες και μη κενό αποτέλεσμα. Στη συνέχεια, το βασικό query συνδέει τον ασθενή με τις νοσηλείες του, τα αντίστοιχα τμήματα, τους κωδικούς KEN και τις διαγνώσεις ICD-10 εισαγωγής και εξόδου. Οι αξιολογήσεις συνδέονται με LEFT JOIN, ώστε να μην αποκλείονται νοσηλείες που δεν έχουν αξιολόγηση.

Το τελικό plan έχει εκτιμώμενο κόστος 20.7 και πραγματικό χρόνο εκτέλεσης περίπου 1.31..1.71 ms, επιστρέφοντας 8 γραμμές. Σε σχέση με το αρχικό plan, όπου ο χρόνος ήταν περίπου 92.4 ms, η βελτίωση είναι σημαντική. Το βασικό συμπέρασμα είναι ότι το κατάλληλο index στον πίνακα hospitalization περιορίζει άμεσα το query στον ζητούμενο ασθενή, ενώ τα υπόλοιπα joins εκτελούνται αποδοτικά με primary key lookups.

```
***** 1. row *****
EXPLAIN: -> Nested loop left join (cost=20.7 rows=8) (actual time=0.144..0.329 rows=8 loops=1)
  -> Nested loop left join (cost=16.8 rows=8) (actual time=0.141..0.31 rows=8 loops=1)
    -> Nested loop left join (cost=14 rows=8) (actual time=0.135..0.282 rows=8 loops=1)
      -> Nested loop inner join (cost=11.2 rows=8) (actual time=0.119..0.238 rows=8 loops=1)
        -> Nested loop inner join (cost=8.4 rows=8) (actual time=0.0948..0.178 rows=8 loops=1)
          -> Nested loop inner join (cost=5.6 rows=8) (actual time=0.0839..0.131 rows=8 loops=1)
            -> Filter: ((h.patient_amka = '30000000202') and <cache>(('30000000202' = (@target_patient_amka)))) (cost=2.8 rows=8) (actual time=0.0395..0.0749 rows=8 loops=1)
              -> Index lookup on h using idx_hosp_patient_admission_q6 (patient_amka = (@target_patient_amka)) (reverse) (cost=2.8 rows=8) (actual time=0.0375..0.0697 rows=8 loops=1)
                -> Single-row index lookup on d using PRIMARY (department_id = h.department_id) (cost=0.263 rows=1) (actual time=0.00656..0.00662 rows=1 loops=8)
                  -> Single-row index lookup on k using PRIMARY (ken_code = h.ken_code) (cost=0.263 rows=1) (actual time=0.00537..0.00543 rows=1 loops=8)
                    -> Single-row index lookup on adm using PRIMARY (icd10_code = h.admission_icd10_code) (cost=0.263 rows=1) (actual time=0.00711..0.00717 rows=1 loops=8)
                      -> Single-row index lookup on disc using PRIMARY (icd10_code = h.discharge_icd10_code) (cost=0.263 rows=1) (actual time=0.0051..0.00516 rows=1 loops=8)
                        -> Single-row index lookup on he using PRIMARY (hosp_id = h.hosp_id) (cost=0.263 rows=1) (actual time=0.00311..0.00318 rows=1 loops=8)
                          -> Filter: (pe.patient_amka = '30000000202') (cost=0.394 rows=1) (actual time=0.00118..0.0016 rows=1 loops=8)
                            -> Table scan on pe (cost=8.45..8.45 rows=1) (actual time=588e-6..853e-6 rows=1 loops=8)
                              -> Materialize CTE patient_eval (cost=5.94..5.94 rows=1) (never executed)
```

Στο βελτιστοποιημένο Q06, το βασικό ευρετήριο που χρησιμοποιείται είναι το idx_hosp_patient_admission_q6 στον πίνακα hospitalization. Το EXPLAIN ANALYZE δείχνει Index lookup on h using idx_hosp_patient_admission_q6, άρα η MySQL εντοπίζει τις νοσηλείες του συγκεκριμένου ασθενή με βάση το patient_amka, χωρίς full table scan. Οι υπόλοιποι πίνακες (department, ken, icd10_diagnosis, hospitalization_evaluation) προσπελούνται μέσω των PRIMARY keys με single-row index lookups, επομένως το κόστος τους είναι μικρό. Το CTE patient_eval δεν χρησιμοποιεί κάποιο custom index στο συγκεκριμένο plan, αλλά γίνεται table scan πάνω σε πολύ μικρό προσωρινό αποτέλεσμα ενός row, οπότε δεν αποτελεί πρόβλημα για την αποδοτικότητά.

Παρακάτω φαίνεται και το runtime του συγκεκριμένου sql file με το force index στο optimal index που θεωρούμε ότι ταιριάζει στις απαιτήσεις της βάσης μας. Παρατηρούμε ότι το query μας τρέχει αρκετά γρήγορα με το extra index που προσθέσαμε τελικά για τις απαιτήσεις της άσκησης.


```

MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q06.sql > sql/Q06_out.txt

real    0m0.138s
user    0m0.028s
sys     0m0.022s
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q06.sql > sql/Q06_out.txt

real    0m0.094s
user    0m0.026s
sys     0m0.016s
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q06.sql > sql/Q06_out.txt

real    0m0.105s
user    0m0.027s
sys     0m0.020s
MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q06.sql > sql/Q06_out.txt

real    0m0.138s
user    0m0.027s
sys     0m0.019s

```

Ο `idx_hosp_patient_admission_q6` είναι λογικά καταλληλότερο για το Q6, επειδή ταιριάζει απευθείας στο φίλτρο `patient_amka` και στο `ORDER BY admission_ts DESC`. Το εναλλακτικό index `idx_hosp_patient_dept_dates` επίσης βρίσκει γρήγορα τις νοσηλείες του ασθενή, αλλά απαιτεί ξεχωριστό sort για το `admission_ts DESC`. Επειδή το αποτέλεσμα περιέχει μόνο 8 νοσηλείες, η πραγματική διαφορά χρόνου είναι μικρή στο συγκεκριμένο dataset, αλλά το plan με το ειδικό Q6 index είναι θεωρητικά καθαρότερο και κλιμακώνεται καλύτερα.

```

MacBook-Pro-Euangelos:Ygeiopolis-healthcare-system euangeloseuangelou$ time mysql -u root -t yg_eupolis_hospital <
sql/Q06.sql > sql/Q06_out.txt
***** 1. row *****
EXPLAIN: -> Nested loop left join (cost=20.7 rows=8) (actual time=0.165..0.293 rows=8 loops=1)
  -> Nested loop left join (cost=16.8 rows=8) (actual time=0.162..0.277 rows=8 loops=1)
    -> Nested loop left join (cost=14 rows=8) (actual time=0.156..0.254 rows=8 loops=1)
      -> Nested loop inner join (cost=11.2 rows=8) (actual time=0.138..0.212 rows=8 loops=1)
        -> Nested loop inner join (cost=8.4 rows=8) (actual time=0.116..0.159 rows=8 loops=1)
          -> Nested loop inner join (cost=5.6 rows=8) (actual time=0.1..0.113 rows=8 loops=1)
            -> Sort: h.admission_ts DESC (cost=2.8 rows=8) (actual time=0.0737..0.0766 rows=8 loops=1)
              -> Index lookup on h using idx_hosp_patient_dept_dates (patient_amka = (@target_patient_amka)), with index condition: ((h.patient_amka = '30000000202') and <cache>('30000000202' = (@target_patient_amka))) (cost=2.8 rows=8) (actual time=0.0587..0.062 rows=8 loops=1)
                -> Single-row index lookup on d using PRIMARY (department_id = h.department_id) (cost=0.263 rows=1) (actual time=0.00416..0.00422 rows=1 loops=8)
                  -> Single-row index lookup on k using PRIMARY (ken_code = h.ken_code) (cost=0.263 rows=1) (actual time=0.00544..0.00549 rows=1 loops=8)
                    -> Single-row index lookup on adm using PRIMARY (icd10_code = h.admission_icd10_code) (cost=0.263 rows=1) (actual time=0.00622..0.00628 rows=1 loops=8)
                      -> Single-row index lookup on disc using PRIMARY (icd10_code = h.discharge_icd10_code) (cost=0.263 rows=1) (actual time=0.00486..0.00491 rows=1 loops=8)
                        -> Single-row index lookup on he using PRIMARY (hosp_id = h.hosp_id) (cost=0.263 rows=1) (actual time=0.00266..0.00266 rows=1 loops=8)
                          -> Filter: (pe.patient_amka = '30000000202') (cost=0.394 rows=1) (actual time=0.00105..0.00139 rows=1 loops=8)
                            -> Table scan on pe (cost=8.45..8.45 rows=1) (actual time=514e-6..746e-6 rows=1 loops=8)
                              -> Materialize CTE patient_eval (cost=5.94..5.94 rows=1) (never executed)

```

Q7:

Το ερώτημα 7 ξεκινά από τον πίνακα `active_substance`, ώστε να εμφανιστούν όλες οι δραστικές ουσίες, ακόμα και όσες δεν έχουν δηλωθεί ως αλλεργίες ή δεν συνδέονται με φάρμακα. Με το `LEFT JOIN` στον `patient_allergy` μετράμε πόσοι διαφορετικοί ασθενείς έχουν δηλώσει αλλεργία σε κάθε δραστική ουσία, ενώ με το `LEFT JOIN` στον `drug_active_substance` μετράμε πόσα διαφορετικά φάρμακα την περιέχουν. Η χρήση του `COUNT(DISTINCT ...)` είναι σημαντική,

γιατί αποφεύγει διπλομετρήσεις που μπορεί να προκύψουν από τα δύο joins. Τέλος, το GROUP BY ομαδοποιεί τα αποτελέσματα ανά δραστική ουσία και το ORDER BY ταξινομεί πρώτα τις ουσίες με τους περισσότερους αλλεργικούς ασθενείς, μετά αυτές που υπάρχουν σε περισσότερα φάρμακα, και τέλος αλφαβητικά με βάση το όνομα της ουσίας.

Q8:

Αυτό το Query εντοπίζει ποια μέλη του προσωπικού του νοσοκομείου ΔΕΝ είχαν βάρδια σε ένα συγκεκριμένο τμήμα κατά τη διάρκεια μιας συγκεκριμένης ημέρας. Συγκεκριμένα, αναζητά το προσωπικό που δεν δούλεψε στο Τμήμα 5 στις 10 Μαρτίου 2026. Μπορούμε να αλλάξουμε την ημερομηνία και το τμήμα στο ίδιο το query.

Το Εσωτερικό Subquery: Μέσα στην παρένθεση, το ερώτημα συνδέει τις αναθέσεις βαρδιών shift_assignment με τις βάρδιες των τμημάτων department_shift. Φιλτράρει τις εγγραφές για να βρει τις βάρδιες που έγιναν στο Τμήμα 5 (department_id = 5) τη συγκεκριμένη ημερομηνία '2026-03-10' και επιστρέφει μια λίστα με τα ΑΜΚΑ των υπαλλήλων που δούλεψαν εκεί.

Το Εξωτερικό Main Query: Ζητάει το όνομα, το επώνυμο, το ΑΜΚΑ και τον τύπο (π.χ. γιατρός, νοσηλεύτης) του προσωπικού από τον πίνακα personnel. Χρησιμοποιώντας την εντολή WHERE amka NOT IN, αποκλείει όλους όσους βρέθηκαν στη λίστα του υποερωτήματος.

Το τελικό αποτέλεσμα είναι μια λίστα με το προσωπικό που ήταν διαθέσιμο, είχε ρεπό, ή δούλεψε σε κάποιο άλλο τμήμα εκείνη την ημέρα.

Q9:

Αυτό το Query βρίσκει ασθενείς που είχαν τον ίδιο μεγάλο αριθμό ημερών νοσηλείας (>15) μέσα στην ίδια χρονιά.

Υπολογισμός ανά έτος (patient_totals): Παίρνει το ιστορικό, ξεχωρίζει τις νοσηλείες ανά έτος και προσθέτει τις ημέρες για κάθε ασθενή. Αν ο ίδιος ασθενής πήγε στο νοσοκομείο το 2024 και το 2025 πχ, βγάζει δύο ξεχωριστά σύνολα.

Φιλτράρισμα διάρκειας (total_days > 15): Κρατάει μόνο όσους έκαναν συνολικά πάνω από 15 ημέρες νοσηλείας μέσα σε ένα έτος.

Έλεγχος σύμπτωσης (IN): Το υποερώτημα ψάχνει να βρει αν ο συνδυασμός «Έτος και Ημέρες» υπάρχει πάνω από μία φορά στη βάση. Έτσι, αν ο Ασθενής Α και ο Ασθενής Β είχαν και οι δύο 18 ημέρες νοσηλείας το 2025, εμφανίζονται στο αποτέλεσμα. Αν ο ένας είχε 18 ημέρες το 2024 και ο άλλος το 2025, απορρίπτονται γιατί το έτος διαφέρει.

Ταξινόμηση (ORDER BY): Ταξινομεί τα αποτελέσματα ανά έτος και ανά ημέρες, ώστε οι ασθενείς που συμπίπτουν να βγαίνουν μαζί στην οθόνη.

Q10:

Αυτό το Query εντοπίζει τα τρία πιο δημοφιλή ζεύγη δραστικών ουσιών που έχουν συνταγογραφηθεί μαζί κατά την ίδια νοσηλεία. Η λογική του βασίζεται στη σύνδεση του πίνακα `prescription_substances` με τον εαυτό του με Self Join μέσω του κοινού κωδικού νοσηλείας `hosp_id`, ώστε να εμφανίζει όλους τους πιθανούς συνδυασμούς φαρμάκων που έλαβε ένας ασθενής ταυτόχρονα. Για την αποφυγή λανθασμένων μετρήσεων, το φίλτρο `ps1.substance_id < ps2.substance_id` παίζει καθοριστικό ρόλο, αφού εμποδίζει μια ουσία να βρεθεί με τον εαυτό της και αποκλείει τα αντίστροφα διπλότυπα ζευγάρια, διασφαλίζοντας ότι ο συνδυασμός ουσίας A και B θα καταγραφεί μόνο μία φορά. Στη συνέχεια, η GROUP BY ομαδοποιεί τα δεδομένα ανά ζεύγος ουσιών, η COUNT(*) καταμετρά τη συχνότητα εμφάνισής τους στη βάση, και η ORDER BY frequency DESC σε συνδυασμό με το LIMIT 3 ταξινομεί τα αποτελέσματα φθίνουσα και απομονώνει αποκλειστικά την πρώτη τριάδα των πιο συχνών συνταγογραφήσεων.

Q11:

Το Query λειτουργεί ως εξής:

Υπολογισμός επεμβάσεων ανά γιατρό: Στο κύριο ερώτημα, συνδέει τους γιατρούς με τις επεμβάσεις τους στο `procedure_event` για το τρέχον έτος (`YEAR(pe.start_ts) = YEAR(CURDATE())`) και μετράει πόσες έκανε ο καθένας (`COUNT(*) AS total_procedures`).

Εύρεση του πρώτου με υποερώτημα: Το υποερώτημα μέσα στην παρένθεση υπολογίζει τις επεμβάσεις όλων των γιατρών για το ίδιο έτος, τις ταξινομεί σε φθίνουσα σειρά (`ORDER BY COUNT(*) DESC`) και με το LIMIT 1 επιστρέφει μόνο τον αριθμό των επεμβάσεων που έκανε ο γιατρός που βρίσκεται στην κορυφή.

Το φίλτρο της διαφοράς (HAVING): Η συνθήκη `total_procedures + 5 <=` (το μέγιστο) μαθηματικά ορθή. Αν στον αριθμό των επεμβάσεων ενός γιατρού προσθέσουμε 5 και το αποτέλεσμα εξακολουθεί να είναι μικρότερο ή ίσο από το ρεκόρ του πρώτου, σημαίνει ότι ο γιατρός αυτός έχει κάνει τουλάχιστον 5 λιγότερες επεμβάσεις από τον πρώτο.

Q12:

Αυτό το Query υπολογίζει τις ανάγκες σε προσωπικό για την εβδομάδα της 10ης Μαρτίου 2026 (Μπορούμε να αλλάξουμε εβδομάδα στο Query), παρουσιάζοντας τα αποτελέσματα ανά τμήμα, ημερομηνία, τύπο βάρδιας και υποκατηγορία εργαζομένου. Αρχικά, το φίλτρο WEEK απομονώνει τις ημέρες που ανήκουν στη συγκεκριμένη εβδομάδα, ενώ τα LEFT JOIN συνδέουν το προσωπικό με τους ειδικούς πίνακες των γιατρών, νοσηλευτών και διοικητικών. Στη συνέχεια, μια δομή CASE δημιουργεί τη δυναμική στήλη `personnel_subclass`, η οποία προσαρμόζεται αυτόματα ανάλογα με τον τύπο του υπαλλήλου, εμφανίζοντας την ειδικότητα για τους γιατρούς, τη βαθμίδα για τους νοσηλευτές και τον ρόλο για το διοικητικό προσωπικό. Τέλος, η GROUP BY ομαδοποιεί τις εγγραφές με βάση όλα τα παραπάνω κριτήρια,

επιτρέποντας στη συνάρτηση COUNT(*) να καταμετρήσει πόσα άτομα από κάθε συγκεκριμένη υποκατηγορία απαιτούνται σε κάθε βάρδια και τμήμα, με το τελικό αποτέλεσμα να ταξινομείται χρονολογικά.

Q13:

Το Query χρησιμοποιεί έναν αναδρομικό προσωρινό πίνακα (WITH RECURSIVE), ο οποίος είναι ένας τρόπος στη SQL για να περιηγηθούμε σε δέντρα ιεραρχίας άγνωστου βάθους. Στο πρώτο σκέλος πριν το UNION ALL, ορίζεται η βάση της αναδρομής, όπου εντοπίζεται ο άμεσος επόπτης κάθε γιατρού (Επίπεδο 1). Στο δεύτερο σκέλος, ο πίνακας συνδέεται επανειλημμένα με τον εαυτό του, ανεβαίνοντας σκαλί-σκαλί την ιεραρχία (δηλαδή βρίσκει τον επόπτη του επόπτη) και αυξάνοντας το επίπεδο ($sh.lvl + 1$), μέχρι να φτάσει στον ανώτατο Διευθυντή που δεν έχει πλέον προϊστάμενο (WHERE d.supervisor_amka IS NOT NULL). Στο τελικό στάδιο, το query κάνει JOIN με τους πίνακες του προσωπικού και των γιατρών για να αντικαταστήσει τα AMKA με τα πραγματικά ονόματα και τους βαθμούς των γιατρών και των εποπτών τους, ταξινομώντας το αποτέλεσμα ανά γιατρό και ανά επίπεδο ιεραρχίας.

Q14:

Το Query δημιουργεί αρχικά έναν προσωρινό πίνακα yearly_admissions που ομαδοποιεί τις εισαγωγές ανά κωδικό ICD-10 και ανά έτος, ενώ εφαρμόζει το φίλτρο HAVING COUNT(*) >= 5 για να κρατήσει μόνο τις διαγνώσεις με τουλάχιστον 5 περιστατικά τη συγκεκριμένη χρονιά. Στο κύριο σκέλος του, χρησιμοποιεί Self Join (σύνδεση του πίνακα με τον εαυτό του ως y1 και y2) πάνω στον ίδιο κωδικό διάγνωσης, επιβάλλοντας δύο πολύ συγκεκριμένες συνθήκες: η $y1.yr = y2.yr + 1$ εξασφαλίζει ότι συγκρίνονται δύο συνεχόμενα έτη, ενώ η $y1.admissions = y2.admissions$ φιλτράρει τα αποτελέσματα ώστε να εμφανιστούν μόνο οι περιπτώσεις όπου ο αριθμός των εισαγωγών παρέμεινε ακριβώς ο ίδιος.

Q15:

Αυτό το Query υπολογίζει την κατανομή των ασθενών κάθε επιπέδου επείγοντος στα επιμέρους τμήματα παραπομπής. Βασίζεται σε ένα CTE, LevelTotals, το οποίο συγκεντρώνει τα δεδομένα ανά επίπεδο επείγοντος, όπως τον συνολικό αριθμό περιστατικών, τον μέσο χρόνο αναμονής και το γενικό ποσοστό νοσηλειών. Ο πίνακας αυτός ενώνεται με τα αναλυτικά στοιχεία των επισκέψεων και των τμημάτων και μετά γίνεται ομαδοποίηση GROUP BY για να διασπάσει την πληροφορία ανά κλινική, χρησιμοποιώντας όμως ως σταθερό παρονομαστή το σύνολο περιστατικών ολόκληρου του επιπέδου ($lt.total_cases_in_level$). Με το COUNT(*) / $lt.total_cases_in_level$ βρίσκουμε τις κατανομές, δείχνοντας το ποσοστό συνεισφοράς του κάθε τμήματος επί του συνολικού πλήθους ασθενών του κάθε επιπέδου επείγοντος.

11.Χρήση AI

Κατά την δημιουργία της παρούσας εργασίας χρησιμοποιήθηκαν εργαλεία Τεχνητής Νοημοσύνης πιο συγκεκριμένα το OpenAI ChatGPT, αποκλειστικά με υποστηρικτικό και συμβουλευτικό ρόλο. Η χρήση τους δεν αντικατέστησε τη δική μας μελέτη και προσπάθεια, τον σχεδιασμό της βάσης, την επιλογή του τελικού σχήματος, την υλοποίηση των SQL scripts ή την επαλήθευση των αποτελεσμάτων. Ο βασικός σχεδιασμός του ER μοντέλου, η μετατροπή του σε σχεσιακό σχήμα, η επιλογή των πινάκων και των συσχετίσεων, η τελική μορφή των queries, καθώς και ο έλεγχος της ορθότητας της βάσης πραγματοποιήθηκαν από εμάς τους ίδιους. Τα εργαλεία AI αξιοποιήθηκαν κυρίως για την καλύτερη οργάνωση της σκέψης μας και για την υποβοήθηση σε επιμέρους στάδια της εργασίας. Στο κομμάτι της τεκμηρίωσης, χρησιμοποιήθηκαν για γλωσσική βελτίωση, αναδιατύπωση τεχνικών παραγράφων και καλύτερη απόδοση εννοιών όπως η κανονικοποίηση, οι περιορισμοί ακεραιότητας, τα indexes, τα triggers, τα views και τα stored procedures. Η τελική επιλογή του περιεχομένου, η ακρίβεια των περιγραφών και η αντιστοίχιση τους με την πραγματική υλοποίηση ελέγχθηκαν από εμάς.

Στο κομμάτι της παραγωγής δεδομένων, τα εργαλεία AI χρησιμοποιήθηκαν βοηθητικά για τη διαμόρφωση της λογικής του data generation, για την οργάνωση της διαδικασίας παραγωγής query-driven synthetic data και για την αποφυγή λαθών σε επιμέρους σημεία του Python script. Τα τελικά δεδομένα δεν εισήχθησαν απευθείας ως αποτέλεσμα κάποιου LLM, αλλά παράχθηκαν μέσω δικού μας script, το οποίο εκτελέστηκε, ελέγχθηκε και προσαρμόστηκε ώστε να είναι συμβατό με το σχεσιακό σχήμα, τα foreign keys, τα triggers και τα ζητούμενα queries. Τα reference δεδομένα, όπως ICD-10, KEN, κατάλογοι ιατρικών πράξεων και δεδομένα φαρμάκων και δραστικών ουσιών, βασίστηκαν στις επίσημες πηγές της εκφώνησης και όχι σε επινοημένο περιεχόμενο. Στο στάδιο της υλοποίησης, τα εργαλεία AI χρησιμοποιήθηκαν επίσης ως βοηθήματα debugging και εκμάθησης. Συγκεκριμένα, μας βοήθησαν να εντοπίσουμε πιθανά συντακτικά προβλήματα σε SQL scripts, να κατανοήσουμε καλύτερα περιορισμούς της MySQL, τη συμπεριφορά των FOREIGN KEY constraints, τη λειτουργία των triggers και τη δομή των stored procedures, καθώς και να βελτιώσουμε την αναγνωσιμότητα ορισμένων query explanations. Παρ' όλα αυτά, κάθε διόρθωση ελέγχθηκε χειροκίνητα στη βάση και έγινε αποδεκτή μόνο εφόσον ήταν συμβατή με το δικό μας schema.

Για την δημιουργία του UI, τα εργαλεία AI χρησιμοποιήθηκαν κυρίως για αρχικές ιδέες οργάνωσης του και για βελτίωση της παρουσίασης των δεδομένων. Η τελική προσαρμογή της εφαρμογής, η σύνδεσή της με τα SQL queries, η χρήση των views και η ασφαλής ανάγνωση των αποτελεσμάτων από τη βάση έγιναν με βάση τις ανάγκες του δικού μας project. Το UI λειτουργεί υποστηρικτικά για την παρουσίαση της βάσης και δεν αντικαθιστά τα SQL παραδοτέα της εργασίας.

Συνολικά, η χρήση των εργαλείων Τεχνητής Νοημοσύνης περιορίστηκε σε βοηθητικές ενέργειες, όπως αναδιατύπωση, επεξήγηση εννοιών, εντοπισμό πιθανών σφαλμάτων, οργάνωση τεκμηρίωσης και υποστήριξη στη διαδικασία ανάπτυξης. Η τελική ευθύνη για τον σχεδιασμό,

την υλοποίηση, την εκτέλεση, τον έλεγχο και την ορθότητα του παραδοτέου ανήκει αποκλειστικά στην ομάδα.

12. Εγκατάσταση βάσης

Η εγκατάσταση της βάσης δεδομένων έχει οργανωθεί έτσι ώστε να μπορεί να εκτελεστεί με επαναλήψιμο και ασφαλή τρόπο από το root directory του repository. Αρχικά απαιτείται εγκατεστημένο περιβάλλον MySQL ή MariaDB, καθώς η υλοποίηση βασίζεται σε SQL scripts.

Η προτεινόμενη διαδικασία είναι η εκτέλεση των έτοιμων runner scripts, δηλαδή του `run_database.sh` σε macOS/Linux ή του `run_database_windows.bat` σε Windows. Τα scripts αυτά αυτοματοποιούν τα βασικά βήματα της εγκατάστασης: ελέγχουν ότι η εκτέλεση γίνεται από το σωστό directory, ενεργοποιούν την επιλογή `local_infile`, δημιουργούν τη βάση και όλα τα αντικείμενα του σχήματος μέσω του `sql/install.sql`, φορτώνουν τα CSV δεδομένα μέσω του `sql/load.sql` και στο τέλος εκτελούν το `sql/validation.sql` για έλεγχο ορθότητας.

Εναλλακτικά, η εγκατάσταση μπορεί να γίνει χειροκίνητα με τις εντολές `mysql -u root -p < sql/install.sql`, `mysql --local-infile=1 -u root -p < sql/load.sql` και `mysql -u root -p < sql/validation.sql`. Το `install.sql` δημιουργεί τη βάση `ygeiopolis`, τους πίνακες, τα κλειδιά, τους περιορισμούς, τα indexes, τα views, τα triggers και τις stored procedures. Το `load.sql` φορτώνει τα δεδομένα από τους φακέλους `data/reference` και `data/generated`, χρησιμοποιώντας relative paths, άρα είναι σημαντικό η εκτέλεση να γίνεται από το root του repository. Τέλος, το `validation.sql` εμφανίζει πλήθη εγγραφών και εκτελεί ελέγχους για πιθανά προβλήματα, όπως ελλειπείς συσχετίσεις, μη έγκυρα foreign keys, επικαλύψεις σε νοσηλείες ή επεμβάσεις και ασυμβατότητες στη συνταγογράφηση. Με αυτή τη διαδικασία, η βάση μπορεί να εγκατασταθεί από την αρχή με σταθερή σειρά, χωρίς χειροκίνητες αλλαγές στα paths ή στα δεδομένα, ενώ η επιτυχία της φόρτωσης επιβεβαιώνεται αμέσως από τα validation queries.

13. Repository Structure

Το repository του project οργανώθηκε με τρόπο ώστε να διαχωρίζονται καθαρά τα διαφορετικά επίπεδα της υλοποίησης: ο σχεδιασμός της βάσης, τα δεδομένα, τα SQL scripts, η παραγωγή δεδομένων, η τεκμηρίωση και η προαιρετική εφαρμογή επίδειξης. Η βασική λογική πίσω από αυτή τη δομή είναι ότι η εργασία δεν αποτελεί μόνο ένα μεμονωμένο αρχείο SQL, αλλά ένα πλήρες database project που πρέπει να μπορεί να εγκατασταθεί, να φορτωθεί με δεδομένα, να ελεγχθεί και να παρουσιαστεί με επαναλήψιμο τρόπο.

Ο φάκελος `diagrams` περιέχει τα διαγράμματα σχεδιασμού της βάσης, δηλαδή το ER διάγραμμα και το σχεσιακό διάγραμμα. Ο ρόλος του είναι να αποτυπώνει οπτικά τη μοντελοποίηση του προβλήματος πριν αυτή μετατραπεί σε SQL υλοποίηση. Το ER διάγραμμα δείχνει τις βασικές

οντότητες, τις σχέσεις και τις πληθικότητες, ενώ το σχεσιακό διάγραμμα αποτυπώνει την τελική μορφή των πινάκων, των primary keys, των foreign keys και των συσχετίσεων. Η ύπαρξη ξεχωριστού φακέλου για τα διαγράμματα είναι σημαντική γιατί ο σχεδιασμός της βάσης αποτελεί ανεξάρτητο παραδοτέο και δεν πρέπει να συγχέεται με τα SQL scripts ή με τα δεδομένα φόρτωσης.

Ο φάκελος data είναι ένας από τους πιο σημαντικούς φακέλους του repository, επειδή περιέχει τα CSV αρχεία που φορτώνονται στη βάση. Χωρίζεται σε δύο υποφακέλους: data/reference και data/generated. Ο διαχωρισμός αυτός είναι σκόπιμος. Στο data/reference/ τοποθετούνται δεδομένα αναφοράς, δηλαδή δεδομένα που προέρχονται από επίσημες ή εξωτερικές πηγές και δεν πρέπει να δημιουργούνται αυθαίρετα. Τέτοια είναι οι ICD-10 διαγνώσεις, τα KEN, ο κατάλογος ιατρικών πράξεων, τα εργαστηριακά tests, τα φάρμακα, οι δραστικές ουσίες και οι σχέσεις φαρμάκων-δραστικών ουσιών. Αντίθετα, στο data/generated αποθηκεύονται τα synthetic operational δεδομένα, όπως ασθενείς, προσωπικό, νοσηλείες, βάρδιες, επείγοντα περιστατικά, συνταγογραφήσεις, αξιολογήσεις και procedure events. Με αυτόν τον τρόπο διατηρείται καθαρή διάκριση ανάμεσα σε επίσημα reference data και σε συνθετικά δεδομένα που δημιουργήθηκαν για να καλύψουν τις ανάγκες των ερωτημάτων.

Ο φάκελος scripts περιέχει το generate_data.py και το σχετικό README. Το αρχείο αυτό δεν είναι απαραίτητο να εκτελεστεί κατά τη διαδικασία εγκατάστασης της βάσης, επειδή το τελικό repository περιλαμβάνει ήδη τα έτοιμα CSV αρχεία. Παρ' όλα αυτά, διατηρείται στο repository για λόγους επαναληψιμότητας και διαφάνειας. Αν χρειαστεί να ξαναδημιουργηθούν τα δεδομένα, το script μπορεί να εκτελεστεί με συγκεκριμένα arguments και να ξαναγράψει τους φακέλους data/reference και data/generated. Η ύπαρξη του generator δείχνει ότι το dataset δεν φτιάχτηκε χειροκίνητα και τυχαία, αλλά με οργανωμένο, deterministic και query-driven τρόπο. Δηλαδή, τα δεδομένα παράγονται έτσι ώστε να σέβονται τους περιορισμούς της βάσης και να επιστρέφουν μη κενά αποτελέσματα στα ζητούμενα queries.

Ο φάκελος sql/ αποτελεί τον πυρήνα της βάσης δεδομένων. Περιέχει το install.sql, το load.sql, το validation.sql, τα αρχεία Q01.sql έως Q15.sql, καθώς και τα αντίστοιχα output files Q01_out.txt έως Q15_out.txt. Το install.sql δημιουργεί τη βάση δεδομένων, τους πίνακες, τα primary keys, τα foreign keys, τα check constraints, τα indexes, τα views, τα triggers και τα stored procedures. Το load.sql φορτώνει τα CSV δεδομένα από τους φακέλους data/reference και data/generated, χρησιμοποιώντας relative paths ώστε το project να είναι portable και να μπορεί να τρέξει από το repository root. Το validation.sql εκτελεί ελέγχους μετά τη φόρτωση, όπως row counts και problem-detection queries, ώστε να επιβεβαιωθεί ότι τα δεδομένα φορτώθηκαν σωστά και δεν παραβιάζονται κρίσιμοι κανόνες. Τα Q01.sql έως Q15.sql περιέχουν τις τελικές απαντήσεις στα ζητούμενα ερωτήματα της εκφώνησης, ενώ τα .txt outputs αποθηκεύουν τα αποτελέσματα της εκτέλεσής τους.

Η λογική πίσω από αυτή την οργάνωση είναι ότι η βάση πρέπει να μπορεί να στηθεί από την αρχή με ξεκάθαρη σειρά. Πρώτα εκτελείται το `install.sql`, ώστε να δημιουργηθεί το σχήμα. Έπειτα εκτελείται το `load.sql`, ώστε να φορτωθούν τα reference και generated δεδομένα. Μετά εκτελείται το `validation.sql`, ώστε να επιβεβαιωθεί ότι η εγκατάσταση είναι σωστή. Τέλος, μπορούν να εκτελεστούν τα queries Q1–Q15. Αυτή η ακολουθία κάνει το project ελέγξιμο. Αν κάτι αποτύχει, είναι σαφές σε ποιο στάδιο βρίσκεται το πρόβλημα: schema creation, data loading, validation ή query execution.

Επιπλέον, το repository περιλαμβάνει runner scripts, όπως `run_database.sh` για macOS/Linux και `run_database_windows.bat` για Windows. Αυτά τα scripts αυτοματοποιούν τη διαδικασία εγκατάστασης, ενεργοποιώντας το `local_infile`, εκτελώντας το `install.sql`, φορτώνοντας τα δεδομένα μέσω `load.sql` και τρέχοντας το `validation.sql`. Η ύπαρξή τους μειώνει την πιθανότητα λάθους κατά την επίδειξη, επειδή ο χρήστης δεν χρειάζεται να πληκτρολογήσει χειροκίνητα όλες τις εντολές MySQL. Ιδιαίτερα στα Windows, το script βοηθά και με την κανονικοποίηση των line endings των CSV αρχείων, ώστε να αποφευχθούν προβλήματα με κρυφούς χαρακτήρες κατά τη φόρτωση.

Στο repository υπάρχει επίσης προαιρετική Streamlit εφαρμογή, η οποία οργανώνεται μέσω των αρχείων `app.py`, `ui.py`, `queries.py`, `streamlit_app.py` και του φακέλου `.streamlit`. Η εφαρμογή δεν αποτελεί τον βασικό τρόπο εγκατάστασης της βάσης, αλλά λειτουργεί ως read-only dashboard για παρουσίαση. Η λογική της είναι να δώσει μια πιο φιλική διεπαφή πάνω από τα SQL queries και τα δεδομένα του νοσοκομείου. Το `app.py` λειτουργεί ως entrypoint, το `ui.py` περιέχει τη διάταξη και τα στοιχεία του interface, ενώ το `queries.py` διαχειρίζεται τη σύνδεση με τη MySQL, την ανάγνωση των predefined query files και την εκτέλεσή τους.